

МЗЯП Ответы на вопросы

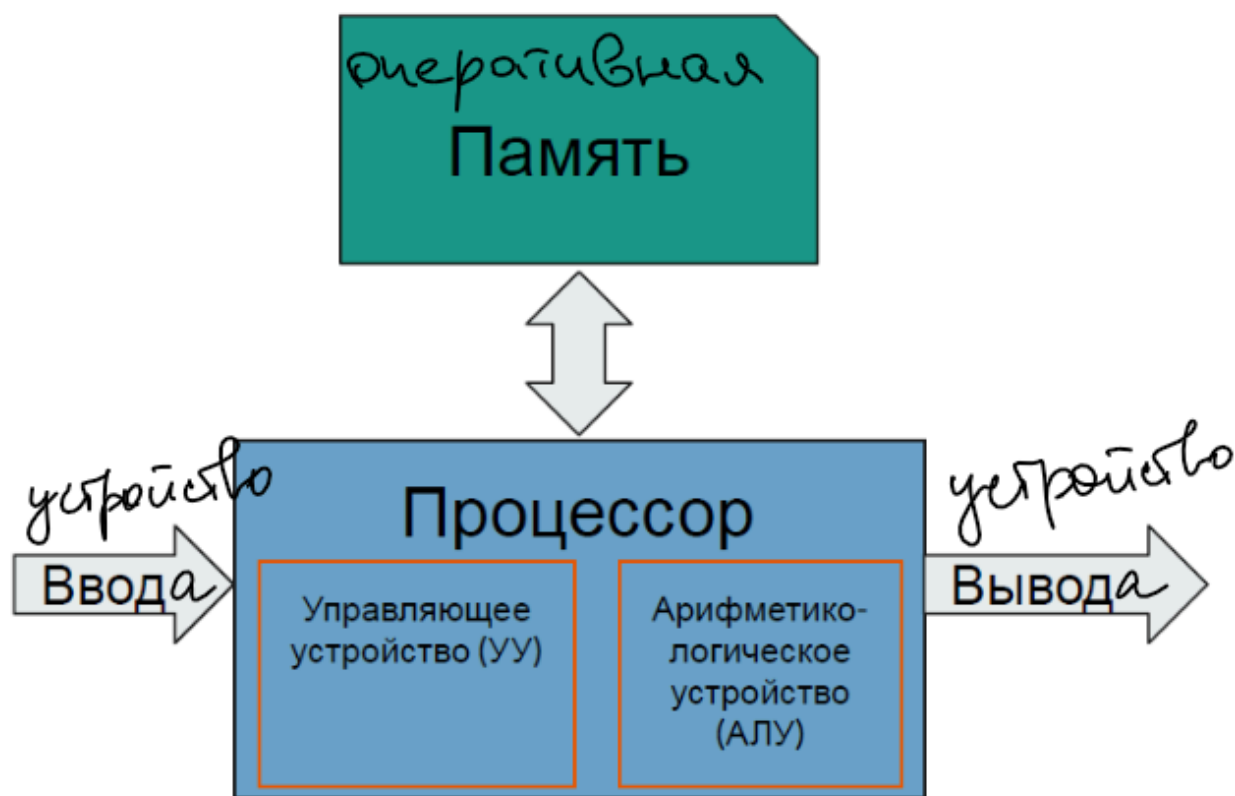
1	Архитектура фон Неймана, принципы фон Неймана.....	3
2	Машинные команды, машинный код. Понятие языка ассемблера.....	5
3	Виды памяти ЭВМ. Запуск и исполнение программы.....	7
4	Сегментная модель памяти в архитектуре 8086.....	9
5	Процессор 8086. Регистры общего назначения.....	11
6	Процессор 8086. Сегментные регистры. Адресация в реальном режиме. Понятие сегментной части адреса и смещения.....	13
7	Процессор 8086. Регистр флагов.....	18
8	Команды пересылки данных.....	20
9	Команда сравнения.....	24
10	Команды условной и безусловной передачи управления.....	25
11	Арифметические команды.....	28
12	Двоично-десятичная арифметика.....	30
13	Команды побитовых операций. Логические команды.....	32
14	Команды работы со строками.....	35
15	Команда трансляции по таблице.....	37
16	Команда вычисления эффективного адреса.....	38
17	Структура программы на языке ассемблера. Модули. Сегменты.....	39
18	Виды ассемблеров. Intel-синтаксис, AT&T-синтаксис.....	42
19	Подпрограммы. Объявление, вызов.....	43
20	Подпрограммы. Возврат управления.....	44
21	Макроопределения.....	45
22	Стек. Аппаратная поддержка вызова подпрограмм.....	46
23	Соглашения о вызовах. Понятие, основные виды соглашений.....	47
24	Прерывания. Обработка прерываний в реальном режиме работы процессора.....	48
25	Процессор 80386. Режимы работы. Регистры.....	51
26	Страничная модель памяти. Виртуальная память.....	56
27	Математический сопроцессор. Типы данных.....	58
28	Математический сопроцессор. Регистры.....	59
29	Математический сопроцессор. Классификация команд.....	60
30	Расширения процессора. MMX. Регистры, поддерживаемые типы данных.....	61
31	Расширения процессора. MMX. Классификация команд.....	63
32	Расширения процессора. SSE. Регистры, поддерживаемые типы данных.....	66

33	Расширения процессора. SSE. Классификация команд.....	68
34	Расширения процессора. AVX. Регистры, поддерживаемые типы данных.....	70
35	Расширения процессора. AVX. Классификация команд.....	72
36	Процессоры семейства x86-64. Регистры, режимы работы.....	74
37	Расширения процессора. AES. Назначение, классификация команд...	75
38	Архитектура RISC. Семейство процессоров ARM. Версии архитектуры, профили.....	77
39	Процессоры ARM. Наборы команд. Основные регистры.....	79
40	Процессоры ARM. Основные команды.....	81

1 Архитектура фон Неймана, принципы фон Неймана

Принципы фон Неймана:

1. Использование **двоичной системы счисления** в вычислительных машинах, что удобнее реализовать (нужно только два состояния)
2. **Программное управление ЭВМ.**
Программа состоит из фиксированного набора вычислительных команд, предусмотренных процессором, которые выполняются процессором друг за другом в определённой последовательности.
3. **Память** компьютера используется не только **для хранения данных, но и программ.** Команды и данные внешне в памяти неразличимы, то есть одно и то же значение в ячейке памяти может использоваться и как данные, и как команда, и как адрес в зависимости лишь от способа обращения к нему.
Следствия:
 - 1) можно использовать модификацию программ (например, циклически изменяя адресную часть команды, обращаться к элементам массива);
 - 2) команды одной программы могут быть получены как результат исполнения другой программы, что лежит в основе процесса трансляции.
4. Ячейки памяти ЭВМ имеют **адреса**, которые **последовательно пронумерованы**, причем процессору в произвольный момент доступна любая ячейка.
5. **Возможность условного перехода** в процессе выполнения программы. При необходимости, с помощью специальных команд, последовательность выполнения команд может быть изменена либо на основании анализа результатов предшествующих вычислений, либо безусловно (по специальной команде).



Гарвардская архитектура является альтернативной. Ее основные принципы:

1. хранилище инструкций и хранилище данных представляют собой разные физические устройства;
2. канал инструкций и канал данных также физически разделены.



2 Машинные команды, машинный код. Понятие языка ассемблера

Машинная команда – инструкция (в двоичном коде) из аппаратно определённого набора, которую способен выполнять процессор.

Машинный код – последовательность машинных команд конкретной вычислительной машины, которая интерпретируется непосредственно процессором.

Язык ассемблера – машинно-зависимый язык программирования низкого уровня, команды которого прямо соответствуют машинным командам.

Компилятор – это специальная программа, которая переводит текст программы, написанный на языке программирования, в набор машинных кодов.

Компоновщик (линковщик, линкер) — инструментальная программа, которая производит компоновку («линковку»): принимает на вход один или несколько объектных модулей и собирает по ним исполняемый модуль.

Отладчик — компьютерная программа для автоматизации процесса отладки: поиска ошибок в других программах. В зависимости от встроенных возможностей, отладчик позволяет выполнять трассировку, отслеживать, устанавливать или изменять значения переменных в процессе выполнения кода, устанавливать и удалять контрольные точки или условия остановки и так далее.

Macro Assembler (MASM) — ассемблер для процессоров семейства x86. Включает компилятор, линковщик и дополнительные инструменты, в т.ч. отладчик. Другие распространённые (сейчас или в прошлом) ассемблеры - TASM, NASM, FASM, GAS, AS и др.

Система команд (также набор команд) — соглашение о

предоставляемых архитектурой средствах программирования, в том числе, типах данных, наборе инструкций, системе регистров и т.д.

Система команд x86 процессора включает в себя следующие шесть основные группы команд:

- команды пересылки данных;
- арифметические команды;
- логические команды;
- команды переходов;
- команды работы с подпрограммами;
- команды управления процессором.

3 Виды памяти ЭВМ. Запуск и исполнение программы

Минимальная адресуемая **единица памяти** – байт.

Машинное слово - машинно-зависимая величина, измеряемая в битах, равная разрядности регистров и шины данных.

Параграф - 16 байт.

Память состоит из отдельных ячеек, имеющих уникальные числовые адреса, называемые **физическими**.

Внутренняя память ЭВМ состоит из ОЗУ (оперативное запоминающее устройство, RAM – Random Access Memory) и ПЗУ (постоянное запоминающее устройство, ROM – Read Only Memory). В ОЗУ можно как записывать, так и считывать информацию, из ПЗУ – только считывать. В последней находится BIOS (Basic Input Output System) и программа начальной загрузки компьютера. Перед выполнением любая программа должна быть загружена в ОЗУ, только после этого процессор начинает считывать и выполнять команды из нее.

Также в ЭВМ может быть внешняя память, предназначенная для хранения долгосрочной информации: жесткие диски (SSD, HDD), дискеты, CD/DVD и т.д..

Загруженная в оперативную память программа выделяет отдельный участок памяти, называемый **стеком**.

Запуск и исполнение программы

Исполняемый файл - файл, содержащий программу в виде, в котором она может быть исполнена компьютером (то есть в машинном коде).

Получение исполняемых файлов обычно включает в себя 2 шага: компиляцию и линковку.

Компилятор - программа для преобразования исходного текста другой программы на определенном языке в объектный модуль.

Компоновщик (линковщик, линкер) - программа для связывания нескольких объектных файлов в исполняемый.

Отладчик – программа для автоматизации процесса отладки. Может выполнять трассировку, отслеживать, устанавливать или изменять значения переменных в процессе выполнения кода, устанавливать или удалять контрольные точки или условия остановки.

В DOS и Windows исполняемые файлы имеют расширения .EXE и .COM

Последовательность запуска программы операционной системой:

1. Определение формата файла.
2. Чтение и разбор заголовка.
3. Считывание разделов исполняемого модуля (файла) в ОЗУ по необходимым адресам.
4. Подготовка к запуску, если требуется (загрузка библиотек).
5. Передача управления на точку входа.

.COM (command) – простейший формат исполняемых файлов DOS и ранних версий Windows:

1. не имеет заголовка;
2. состоит из одной секции, не превышающей 64 Кб;
3. загружается в ОЗУ без изменений;
4. начинает выполняться с 1-го байта (точка входа всегда в начале).

.EXE (executable) файл состоит из заголовка и загружаемой части (тела) файла. Заголовок включает в себя такую информацию, как количество страниц в файле, количество байт в последнем файле, начальное значение регистра SP и так далее.

4 Сегментная модель памяти в архитектуре 8086

В сегментной модели памяти физический адрес формируется из двух частей: **сегментной** и **смещения**.

Сегментом называется условно выделенная область адресного пространства определенного размера, а смещением – адрес ячейки относительно начала сегмента. Вычисление физического адреса выполняется процессором аппаратно, без участия программиста.

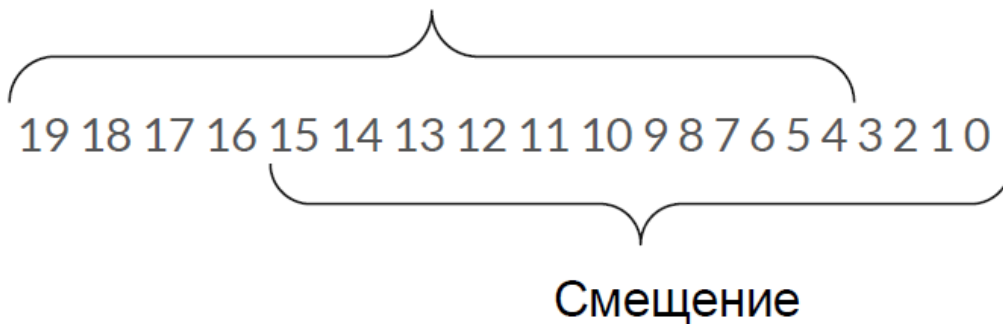
База сегмента — адрес, указывающий на начало сегмента в адресном пространстве.

[SEG]:[OFFSET] => физический адрес:

1. SEG необходимо побитово сдвинуть на 4 разряда влево (или умножить на 16, что тождественно)
2. К результату прибавить OFFSET

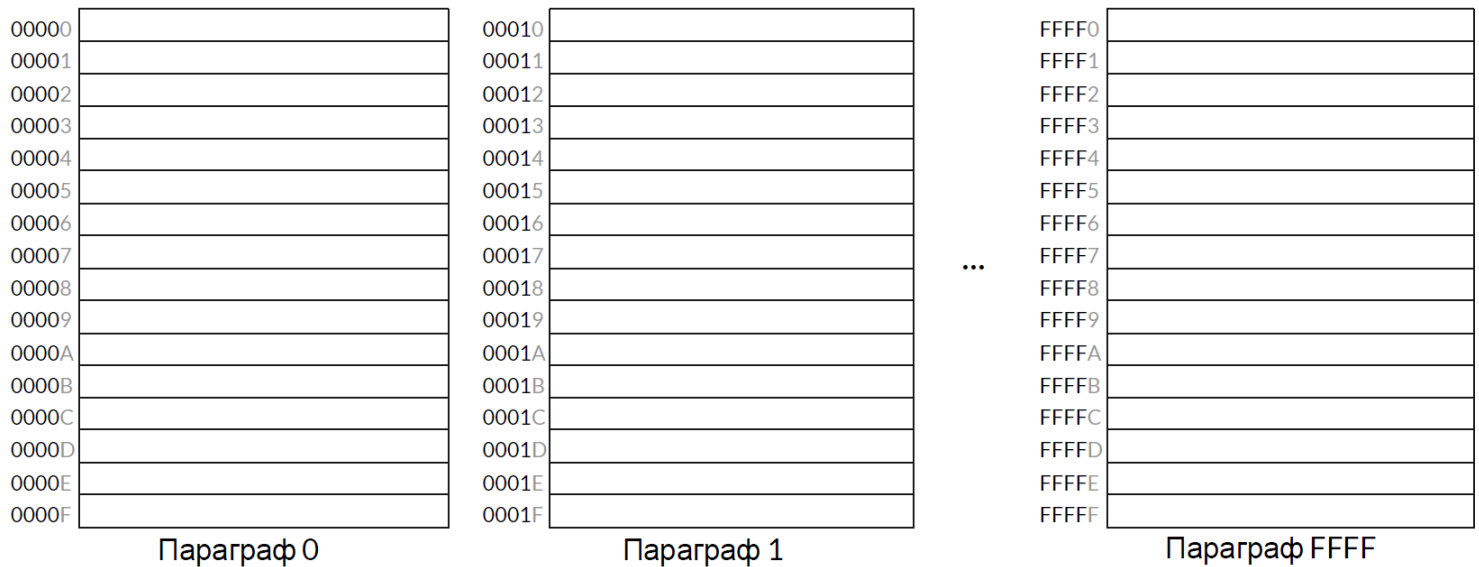
Распространённые пары регистров: CS:IP, DS:BX, SS:SP

Номер параграфа начала сегмента

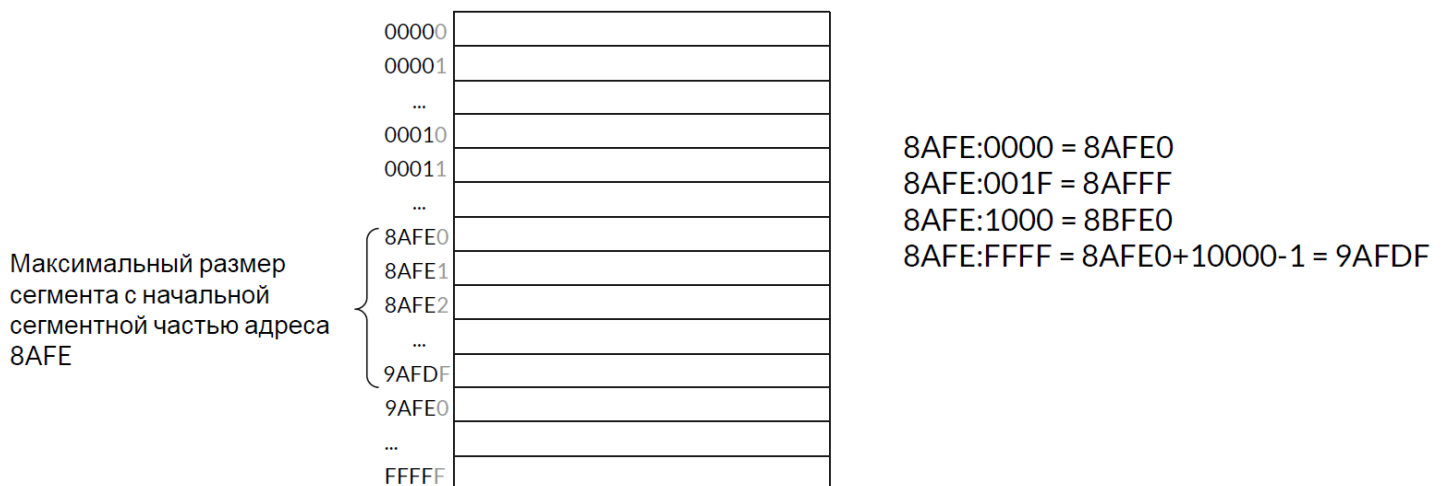


Разрядность шины адреса – 20 разрядов, т.е. доступен 1 Мб (2^{20}) разрядов

Память 8086 (20-разрядная адресация)



Сегментная модель памяти 8086



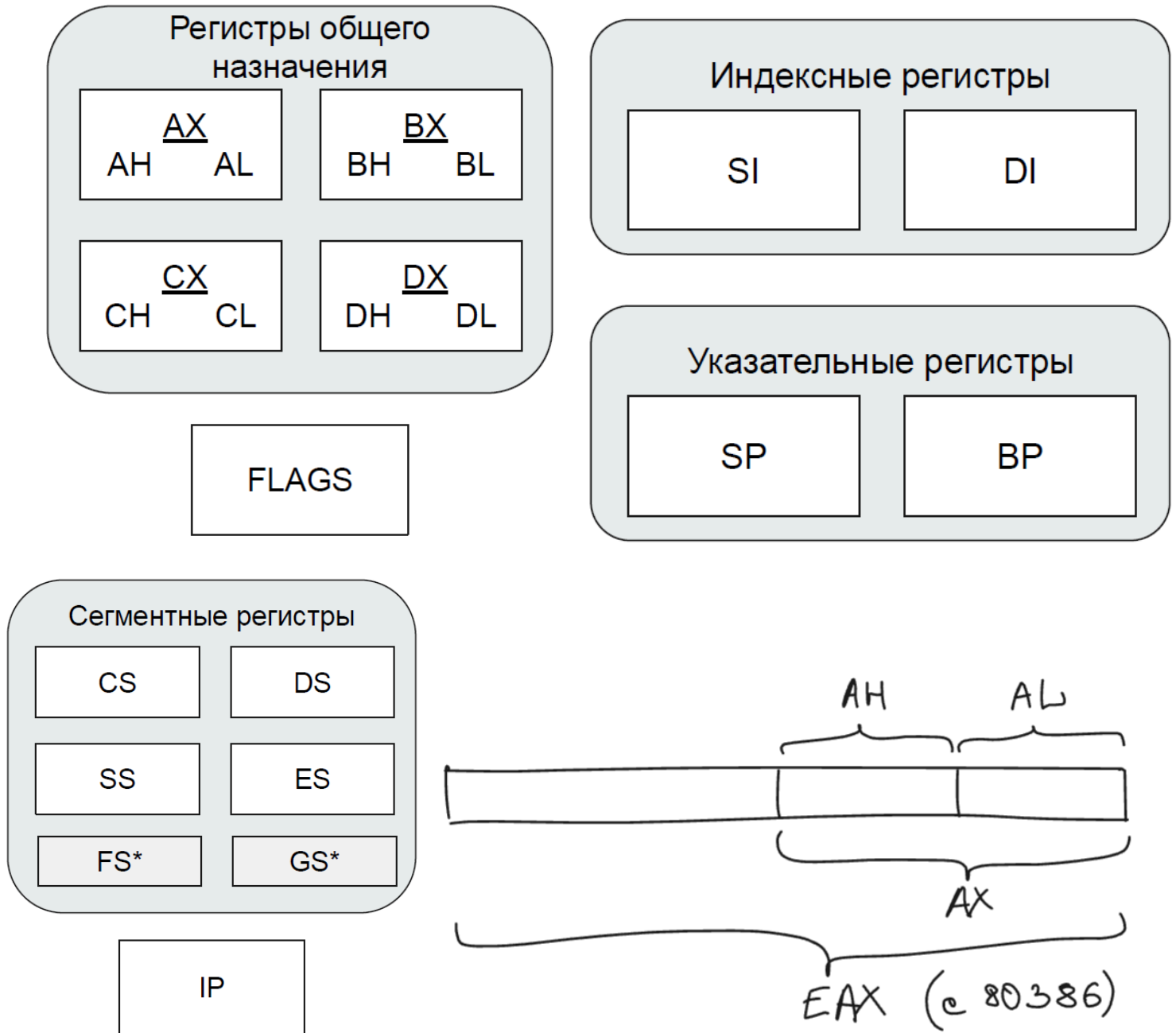
Структура памяти программы:

- Сегмент кода — область памяти, содержащая код программы. За её адресацию отвечает регистр CS. Командой MOV изменить невозможно, меняется автоматически по мере выполнения команд.
- Сегмент данных — область памяти, содержащая переменные и константы программы. Основной регистр для адресации к сегменту данных — DS, при необходимости дополнительных сегментов данных задействуются ES, FS, GS.
- Сегмент стека. Для адресации стека используется регистр SS.

5 Процессор 8086. Регистры общего назначения

Микропроцессор 8086: 16-разрядный, появился в 1978 году, частота 5-10 МГц, техпроцесс 3000 нм (размер транзисторов)

Архитектура 8086 с точки зрения программиста (структура блока регистров)



Регистры общего назначения:

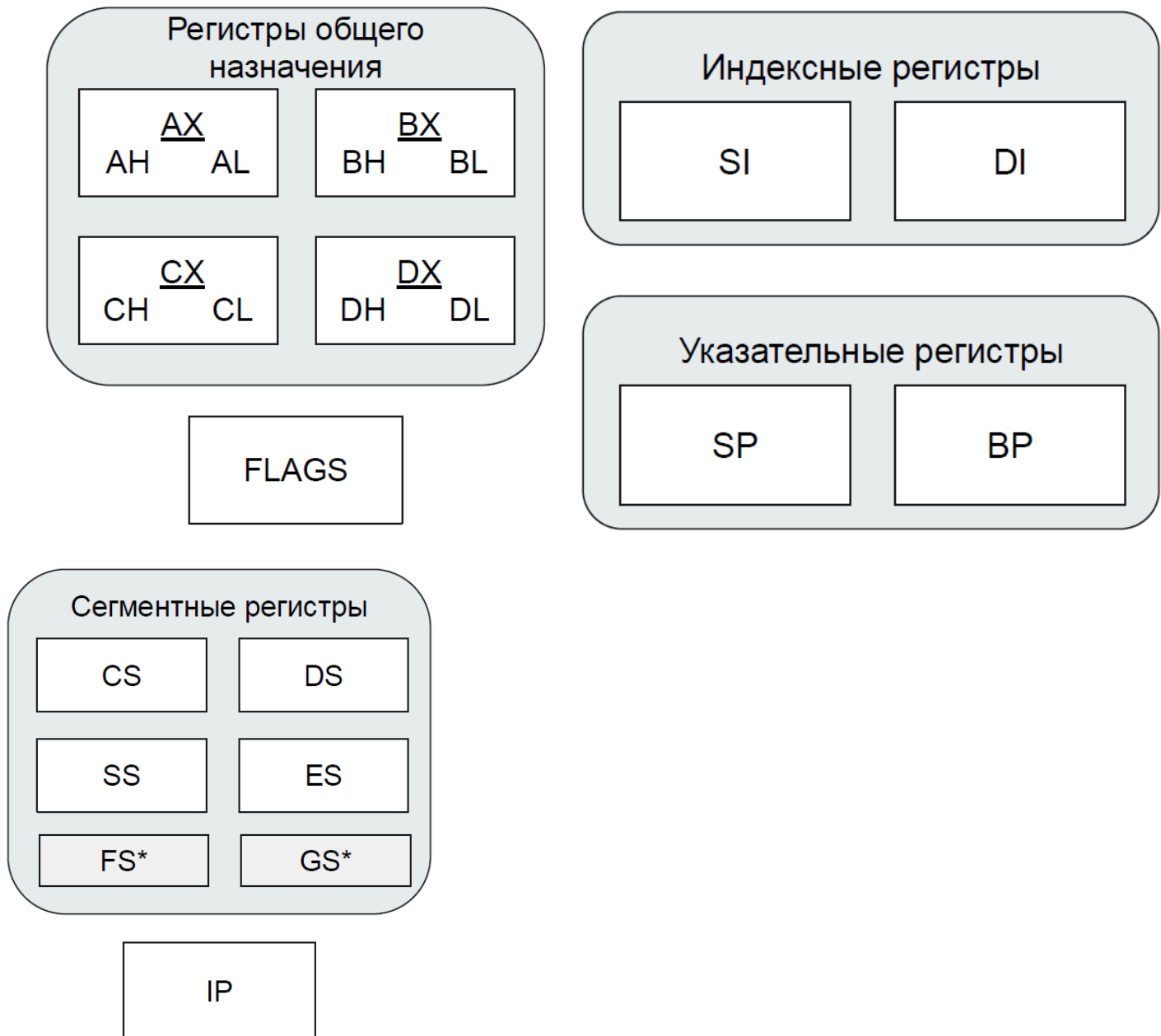
1. AX (accumulator register) — в основном используется для хранения промежуточных данных, обязателен для некоторых команд (int 21h)
2. BX (base register) — в основном используется для хранения промежуточных данных, а также для адресации по базе ([BX] — это адрес DS:BX)

3. CX (count register) — используется для хранения промежуточных данных и для организации циклов
4. DX (data register) — в основном используется для хранения промежуточных данных, обязателен для некоторых команд (при умножении в DX – первая часть результата, в AX – вторая)

6 Процессор 8086. Сегментные регистры. Адресация в реальном режиме. Понятие сегментной части адреса и смещения.

Микропроцессор 8086: 16-разрядный, появился в 1978 году, частота 5-10 МГц, техпроцесс 3000 нм (размер транзисторов)

Архитектура 8086 с точки зрения программиста (структура блока регистров)



Сегментные регистры:

1. CS – сегмент, хранящий адрес начала сегмента кода
2. DS – сегмент, хранящий адрес начала сегмента данных
3. SS – сегмент, хранящий адрес начала сегмента стека
4. ES – дополнительный регистр сегмента данных
5. FS (с 80836) – дополнительный регистр сегмента данных

Адресация

I. Непосредственная адресация.

Операнд может указываться непосредственно в тексте программы, например:

`mov ax, 2` ; здесь 2 задается непосредственно

II. Регистровая адресация.

Операнды могут располагаться в любых регистрах РОН и сегментных регистрах, например:

`mov ax, bx` ; здесь содержимое bx копируется в ax

III. Прямая адресация (адресация по смещению).

Абсолютный адрес можно указывать в формате

СЕГМЕНТ:СМЕЩЕНИЕ. При этом если используют директивы определения данных (ASSUME), то можно ссылаться на статические переменные не по адресу, а по имени. Если сегментный адрес сегмента данных находится в DS, то имя сегментного регистра можно не указывать, DS используется по умолчанию. Примеры:

`mov ax, es:0001`

`mov ax, es:word_var`

IV. Регистровая косвенная адресация

Адрес операнда в памяти можно также не указывать, а хранить в регистре. До процессора 80386 только в BX, SI, DI, BP, затем ограничения были сняты в пользу EAX, EBX, ECX, EDX, ESI, EDI, EBP и ESP. DS используется по умолчанию почти всегда. Но если смещение находится в ESP, EBP или BP, то сегментным регистром должен быть SS. Пример:

`mov ax, [bx]` ; слово из ячейки памяти, сегментный адрессегмента которой находится в DS, а смещение в BX,

помещается в регистр AX

V. Адресация по базе со сдвигом

Комбинация предыдущих двух методов адресаций, пример:

`mov ax, [bx+2]` ; слово, которое есть в сегменте, указанном в DS, со смещением на два больше, чем число из BX, помещается в AX. Такая форма применяется, когда в регистре находится адрес начала структуры данных (массив), а доступ нужен к какому-то ее элементу.

Еще один вариант применения – доступ из подпрограмм к параметрам в стеке, используется BP, сегментная часть указана в SS. Альтернативный способ записи:

```
mov ax, [bp]+2
mov ax, 2[bp]
```

VI Адресация по базе с индексированием

В этом методе адресации смещение операнда в памяти вычисляется как сумма чисел, содержащихся в двух регистрах, и смещения, если оно указано. Ниже разные формы записи одного и того же действия:

```
mov ax, [bx+si+2]
mov ax, [bx][si]+2
mov ax, [bx+2][si]
mov ax, [bx][si+2]
mov ax, 2[bx][si]
```

Смещение равно сумме чисел, содержащихся в BX и SI и числа 2. Можно использовать для доступа к элементам матрицы.

VII Адресация по базе с масштабированием и смещением

Самая полная схема адресации.

Смещение может быть байтом или двойным словом. Если ESP или EBP используются в роли базового регистра, то сегмент

операнда берется по умолчанию из SS, в остальных случаях из DS. Пример:

```
mov ax, [bx][si*4]+10h
```

	<i>EAX</i>				
<i>CS:</i>	<i>EBX</i>	<i>EAX</i>			
<i>SS:</i>	<i>ECX</i>	<i>EBX</i>			
<i>DS:</i>	<i>EDX</i>	<i>ECX</i>	1		
<i>ES:</i>	<i>EBP</i>	<i>EDX</i>	2		
<i>FS:</i>	<i>ESP</i>	<i>EBP</i>	4	+	<i>смещение</i>
<i>GS:</i>	<i>EDI</i>	<i>ESI</i>	8		
	<i>ESI</i>	<i>EDI</i>			

Сегментная часть и смещение

В сегментной модели памяти физический адрес формируется из двух частей: **сегментной** и **смещения**.

Сегментом называется условно выделенная область адресного пространства определенного размера, а смещением – адрес ячейки относительно начала сегмента. Вычисление физического адреса выполняется процессором аппаратно, без участия программиста.

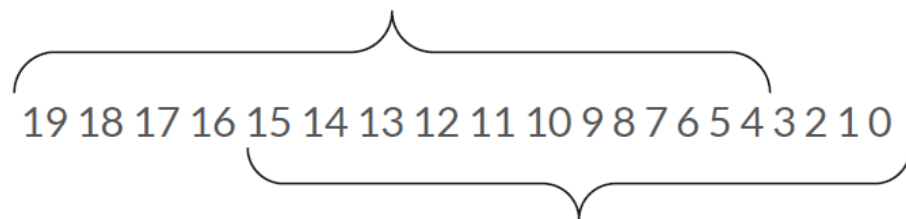
База сегмента — адрес, указывающий на начало сегмента в адресном пространстве.

[SEG]:[OFFSET] => физический адрес:

3. SEG необходимо побитово сдвинуть на 4 разряда влево (или умножить на 16, что тождественно)
4. К результату прибавить OFFSET

Распространённые пары регистров: CS:IP, DS:BX, SS:SP

Номер параграфа начала сегмента



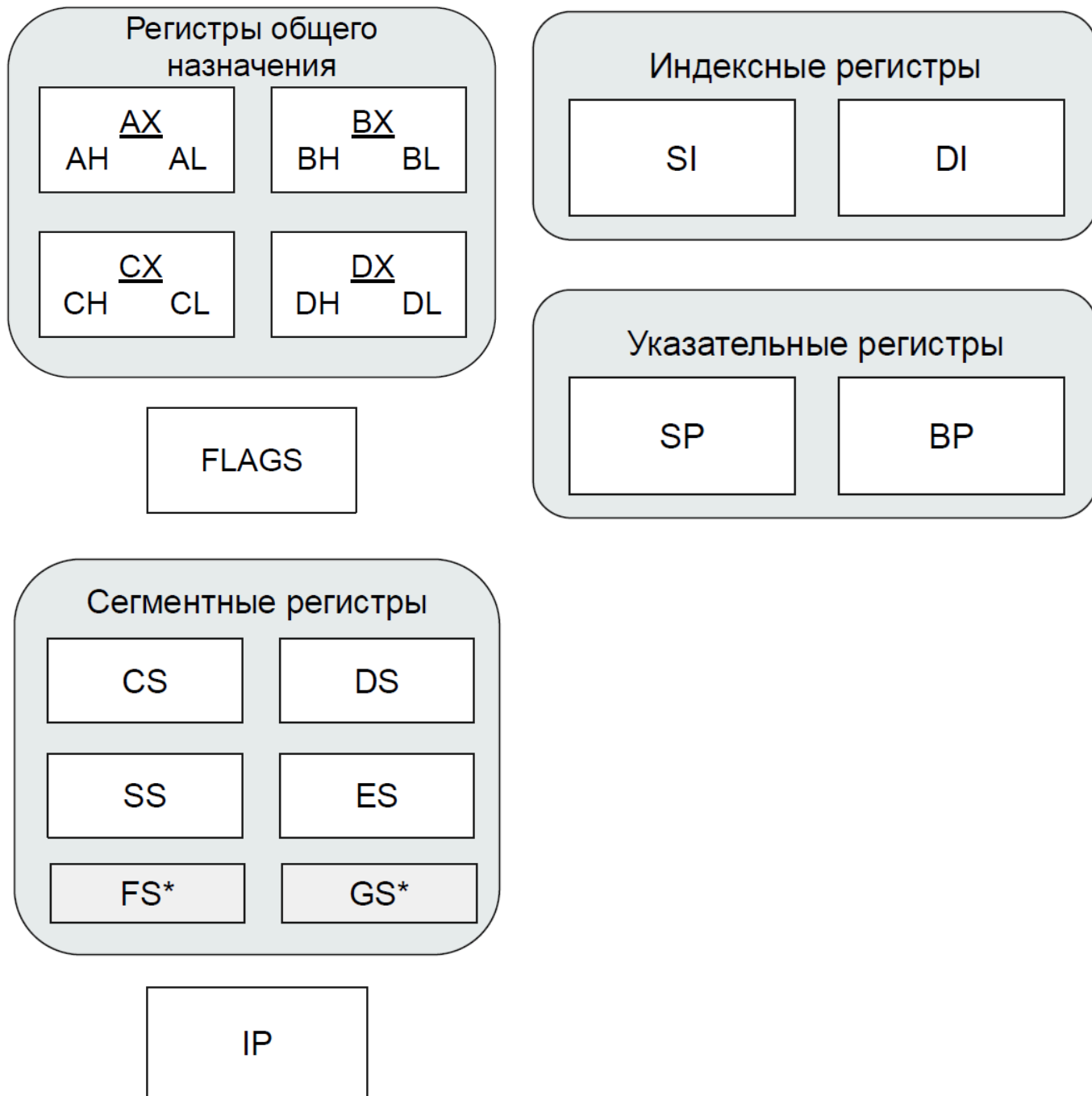
Смещение

Разрядность шины адреса – 20 разрядов, т.е. доступен 1 Мб (2^{20}) разрядов

7 Процессор 8086. Регистр флагов

Микропроцессор 8086: 16-разрядный, появился в 1978 году, частота 5-10 МГц, техпроцесс 3000 нм (размер транзисторов)

Архитектура 8086 с точки зрения программиста (структура блока регистров)



Каждый бит регистра FLAGS является флагом. Младшие 16 бит флага EFLAGS также называются FLAGS, старшие управляют защищенным режимом.



- CF (carry flag) – флаг переноса. **1**, если результат не уместился в приемнике и произошел перенос из старшего бита или требуется заем (при вычитании)
- PF (parity flag) – флаг четности. **1**, если младший байт результата содержит четное число битов, равных **1** (это не делимость на 2)
- AF (auxiliary flag) – флаг вспомогательного переноса. **1**, если в результате предыдущей операции произошел перенос или заем из третьего бита в четвертый. Используется командами двоично-десятичной коррекции автоматически
- ZF (zero flag) – флаг нуля. **1**, если результат предыдущей команды **0**
- SF (sign flag) – флаг знака. Всегда равен старшему биту результата
- TF (trace flag) – флаг ловушки. Предусмотрен для работы отладчиков, не использующих защищенный режим. Если установить **1**, то после каждой команды управление передается отладчику (вызывается прерывание 1)
- IF (interrupt enable flag) – флаг прерываний. Сброс в **0** приводит к тому, что процессор перестает обрабатывать прерывания от внешних устройств.
- DF (direction flag) – флаг направления. Контролирует поведение команд обработки строк: **1**, если обработка в сторону уменьшения адресов; **0**, если наоборот
- OF (overflow flag) – флаг переполнения. **1**, если результат предыдущей арифметической операции над числами со знаком выходит за допустимые для них пределы. Например, если при сложении двух положительных чисел получается число со старшим битом, равным **1**, т.е. отрицательное, и наоборот
- IOPL (input/output privilege level) – флаг определяет уровень привилегии кода, используется в защищенном режиме.
- NT (nested task) – флаг используется в защищенном режиме для фиксации вложенности задач.

8 Команды пересылки данных.

1) Базовая команда пересылки данных:

MOV *приемник, источник*

Источник: число (непосредственно операнд), РОН, сегментный регистр или переменная (операнд, находящийся в памяти)

Приемник: РОН, сегментный регистр (кроме CS) или переменная

Оба операнда должны быть одно и того же размера: байт, слово или двойное слово.

Нельзя: выполнять пересылку из одной переменной в другую, из сегментного регистра в сегментный и помещать в сегментный непосредственный операнд – эти операции выполняют двумя командами.

2) Условная команда пересылки данных:

CMOVcc *приемник, источник*

Пересылка данных выполняется только если выполняется условие cc:

Команда	Условие для CMP	Значения флагов
a nbe	Если выше Если не ниже и не равно	CF=0 и ZF=0
ae nb jnc	Если выше или равно Если не ниже Если перенос	CF=0
b nae c	Если ниже Если не выше и не равно Если перенос	CF=1
be na	Если ниже или равно Если не выше	CF=1 и ZF=1
cxz ecxz	Если CX=0 Если ECX=0	Не влияет на флаги
e z	Если равно Если ноль	ZF=1

g	Если больше	ZF=0 и SF=OF
nle	Если не меньше и не равно	
ge	Если больше или равно	SF=OF
nl	Если не меньше	
l	Если меньше	SF<OF
nge	Если не больше и не равно	
le	Если меньше или равно	ZF=1 или SF<OF
ng	Если не больше	
ne	Если не равно	ZF=0
nz	Если не ноль	
no	Если нет переполнения	OF=0
np	Если нет четности	PF=0
po	Если нечетное	
ns	Если нет знака	SF=0
p	Если есть четность	PF=1
pe	Если четное	
o	Если есть переполнение	OF=1
s	Если есть знак	SF=1

Слова “выше”, “ниже” относятся к сравнению чисел без знака, слова “больше” и “меньше” учитывают знак.

3) Команда обмена

XCHG операнд 1, операнд 2

Содержимое операнда 2 копируется в операнд 1, а старое содержимое операнда 1 – в операнд 2. Можно выполнять над двумя регистрами или над регистром и переменной:

`xchg      eax, ebx`

4) Помещение данных в стек

PUSH источник

Помещает содержимое источника (РОН, сегментный регистр, непосредственный операнд, переменная) в стек. Уменьшает SP на размер источника и копирует содержимое источника в память по адресу SS:[SP].

5) Считывание данных из стека

POP *приемник*

Помещает слово или двойное слово, находящееся в вершине стека, увеличивая SP на 2 или 4 соответственно, в приемник (РОН, сегментный регистр, кроме CS, переменная). Команда RET загружает из стека в CS.

6) Помещение всех РОН в стек

PUSHA

Располагает в стеке регистры в следующем порядке: AX, CX, DX, BX, SP, BP, SI, DI

7) Выгрузка из стека все РОН

POPA

Загружает из стека DI, SI, BP, увеличивает SP на два, загружает BX, DX, CX, AX.

8) Трансляция в соответствии с таблицей.

XLAT *адрес*

Помещает в AL байт из таблицы по адресу ES:BX со смещением относительно начала таблицы равным AL. Адрес, указанный в исходном коде, не обрабатывается компилятором и служит в качестве комментария. Если комментарий не нужен, можно написать XLATB.

Эта команда преобразует байт в соответствии с таблицей преобразований, поэтому часто используется для шифрования методом подстановки.

9) Вычисление эффективного адреса

LEA приемник, источник

Вычисляет эффективный адрес (смещение?) источника (переменная) и помещает его в приемник (регистр). С помощью LEA можно вычислить адрес переменной, которая описана сложным методом адресации.

9 Команда сравнения.

CMP приемник, источник

Вычитает источник из приемника, результат никуда не сохраняется, выставляются флаги CF, PF, AF, ZF, SF, OF

Источник: число, регистр, переменная

Приемник: регистр или переменная, не может быть переменной одновременно с регистром

Обычно используют с командой условного перехода Jcc, условной пересылки данных CMOVcc или условной установки байтов SETcc, позволяющие применить результат сравнения.

CMPXCHG приемник, источник

Начиная с процессора 80486:

Сравнивает значения, содержащиеся в AL, AX, EAX (в зависимости от размера операндов), с приемником (регистром или переменная). Если они равны, информация из источника копируется в приемник и флаг ZF=1. Иначе содержимое приемника копируется в AL, AX, EAX с флаг ZF=0. Остальные флаги определяются по результату операции сравнения, как после CMP.

CMPSB, CMPSW, CMPSD

Сравнивает один байт (CMPSB), слово (CMPSW) или двойное слово (CMPSD) из памяти по адресу DS:ESI (DS:SI) с байтом, словом и двойным словом по адресу ES:EDI (ES:DI) и устанавливает флаги аналогично CMP.

CMPS приемник, источник

Применяя к операндам, регистр DS можно заменить на ES; GS; FS; SS. ES заменить нельзя.

10 Команды условной и безусловной передачи управления.

Команда **безусловной передачи данных**:

JMP операнд

- Передает управление в другую точку программы (на другой адрес памяти), не сохраняя какой-либо информации для возврата.
- Операнд — непосредственный адрес (вставленный в машинный код), адрес в регистре или адрес в переменной.

Типы перехода:

- short (короткий) – если адрес перехода находится в пределах -128...+127 байт от команды JMP.
- near (ближний) – если адрес перехода находится в том же сегменте памяти, что и команда JMP.
- far (дальний) – если адрес перехода находится в другом сегменте. Дальний переход может выполняться и в тот же самый сегмент при условии, что в сегментной части операнда указано число, совпадающее с текущим значением CS.

В случае short и near:

команда фактически преобразовывает значение регистра IP, изменяя тем самым смещение следующей исполняемой команды относительно начала сегмента кода. Если операнд – регистр или переменная в памяти, то его значение просто копируется в IP как при MOV. Если операнд – непосредственно указанное число, то его значение суммируется с содержимым IP, приводя к относительному переходу.

В случае far:

команда просто загружает новое значение в IP и новый сегментный адрес сегмента кода в CS.

Команда **условной передачи данных**:

Jcc метка

Набор команд, выполняющих переход short или near, если удовлетворяется соответствующее условие cc. Когда команда из набора Jcc применяется сразу после CMP, условия приобретают формулировки, соответствующие отношениям между операндами CMP.

Слова “выше”, “ниже” относятся к сравнению чисел без знака, слова “больше” и “меньше” учитывают знак.

Команда	Условие	Значения флагов
a nbe	Если выше Если не ниже и не равно	CF=0 и ZF=0
ae nb jnc	Если выше или равно Если не ниже Если перенос	CF=0
b nae c	Если ниже Если не выше и не равно Если перенос	CF=1
be na	Если ниже или равно Если не выше	CF=1 и ZF=1
cxz ecxz	Если CX=0 Если ECX=0	Не влияет на флаги
e z	Если равно Если ноль	ZF=1

g nle	Если больше Если не меньше и не равно	ZF=0 и SF=OF
ge nl	Если больше или равно Если не меньше	SF=OF
l nge	Если меньше Если не больше и не равно	SF<OF
le ng	Если меньше или равно Если не больше	ZF=1 или SF<OF
ne nz	Если не равно Если не ноль	ZF=0
no	Если нет переполнения	OF=0
np po	Если нет четности Если нечетное	PF=0
ns	Если нет знака	SF=0
p pe	Если есть четность Если четное	PF=1
o	Если есть переполнение	OF=1
s	Если есть знак	SF=1

11 Арифметические команды.

Основные команды целочисленной арифметики:

- *ADD* *приемник, источник* – сложение
Приемник: регистр или переменная
Источник: число, регистр или переменная
Нельзя использовать переменную одновременно и там, и там
- *SUB* *приемник, источник* – вычитание
Приемник: регистр или переменная
Источник: всегда регистр
- *IMUL* *источник* – умножение со знаком, источник умножается на AL, AX, EAX и результат сохраняется там же
IMUL *приемник, источник* – источник (число, регистр или переменная) умножается на приемник (регистр) и результат заносится в приёмник
IMUL *приемник, источник 1. источник 2* – источник 1 (регистр или переменная) умножается на источник 2 (число) и результат заносится в приемник (регистр)
- *MUL* *источник* – выполняет умножение содержимого источника (регистр или переменная) и регистр AL, AX, EAX и результат помещается туда же
- *IDIV* *источник* – целочисленное деление со знаком AL, Ax, EAX на источник (регистр или переменная) и помещает результат в AL, AX, EAX, а остаток в AH, DX, EDX. Результат всегда округляется в сторону нуля.
- *DIV* *источник* – целочисленное деление без знака AL, AX, EAX на источник (регистр или переменная) и помещает результат в AL, AX, EAX, а остаток в AH, DX, EDX. Результат всегда округляется в сторону нуля.
- *INC* *приемник* – увеличение приемника (регистр или переменная) на 1. Флаг CF не затрагивается
- *DEC* *приемник* – уменьшение приемника (регистр или переменная) на 1. Флаг CF не затрагивается

- *NEG* приемник – выполняется над приемником (регистр или переменная) операцию дополнения до двух (обращение знака, операнда)

12 Двоично-десятичная арифметика.

Неупакованное двоично-десятичное число – байт, принимающий значения от 00 до 09h.

Упакованное двоично-десятичное число – байт, принимающий значения от 00 до 99h, не задействуя цифры A, B, C, D, E, F

При выполнении арифметических операций над такими числами получается неверный результат: $19h + 1 = 1A$, а не $20h$. Поэтому требуется коррекция, осуществляемая специальными командами:

- DAA – коррекция сразу после сложения (ADD, ADC, INC, XADD) двух упакованных двоично-десятичных чисел, результат которого находится в AL:
 - 1) Если младшие четыре бита AL больше 9 или флаг AF=1 (результат больше 15, перенос из 3-его бита в 4-ый), то AL увеличивается на 6, CF=1, если был перенос и AF=1
 - 2) Иначе AF=0
 - 3) Если старшие четыре бита AL больше 9 или флаг CF=1, то AL увеличивается на 60h и CF = 1
 - 4) Иначе CF=0SF, ZF, PF устанавливаются в соответствии с результатом, OF не определен

Пример:

inc al

daa

- DAS – коррекция сразу после вычитания (SUB, SBB, DEC) двух упакованных двоично-десятичных чисел, результат которого находится в AL:

Аналогичный алгоритм

Пример:

dec al

das

Пример:

компактный вариант преобразования шестнадцатеричной

цифры в ASCII код соответствующего символа (через XLAT очевиднее, но длиннее)

cmp al, 10

sbb al, 69h

das

- AAA – коррекция сложения неупакованных чисел. Если результат корректировки превышает 9, то задействуется AH и устанавливаются флаги CF и OF
- AAS – коррекция вычитания неупакованных чисел. Если результат корректировки превышает 9, то задействуется AH и устанавливаются флаги CF и OF
- AAM – коррекция умножения неупакованных чисел, устанавливаются флаги SF, ZF, PF. Флаги OF, CF, AF не задействуются
- AAD – коррекция перед делением неупакованных чисел, устанавливаются флаги SF, ZF, PF. Флаги OF, CF, AF не задействуются

13 Команды побитовых операций. Логические команды.

Логические команды:

- *AND* *приемник, источник* – логическое И
Приемник: регистр или переменная
Источник: число, регистр или переменная
Источник и приемник не могут быть переменными одновременно
Флаги OF, CF=0, SF,ZF,PF соответствуют результату, AF нет определен
- *OR* *приемник, источник* – логическое ИЛИ
Приемник: регистр или переменная
Источник: число, регистр или переменная
Источник и приемник не могут быть переменными одновременно
Флаги OF=CF=0, SF,ZF,PF соответствуют результату, AF нет определен
- *XOR* *приемник, источник* – логическое исключающее ИЛИ
Приемник: регистр или переменная
Источник: число, регистр или переменная
Источник и приемник не могут быть переменными одновременно
- *NOT* *приемник* – каждый бит приемника (регистр или переменная) инвертируется. Флаги не затрагиваются.

Команды побитовых операций (с 80836):

- BT база, смещение — команда считывает в флаг CF значение бита из битовой строки, определенной первым операндом – *битовой базой* (число или регистр), со смещением, указанным во втором операнде – *битовом смещении* (число или регистр).
- BTS база, смещение — устанавливает бит в 1 (bit test set)
- BTR база, смещение — сбрасывает бит в 0 (bit test reset)

- BTC база, смещение — инвертирует бит (bit test and complement)
- BSF приемник, источник — сканирует источник (регистр или переменная), начиная с самого младшего бита и записывает в приемник (регистр) номер первого бита, равного 1
- BSR приемник, источник — сканирует источник (регистр или переменная), начиная с самого старшего бита и записывает в приемник (регистр) номер первого бита, равного 1
- SETсс приемник — установка байта приемника (8-битный регистр или переменная) в 1 или 0 в зависимости от выполненного условия

Команда	Условие	Значения флагов
a nbe	Если выше Если не ниже и не равно	CF=0 и ZF=0
ae nb jnc	Если выше или равно Если не ниже Если перенос	CF=0
b nae c	Если ниже Если не выше и не равно Если перенос	CF=1
be na	Если ниже или равно Если не выше	CF=1 и ZF=1
cxz ecxz	Если CX=0 Если ECX=0	Не влияет на флаги
e z	Если равно Если ноль	ZF=1

g nle	Если больше Если не меньше и не равно	ZF=0 и SF=OF
ge nl	Если больше или равно Если не меньше	SF=OF
l nge	Если меньше Если не больше и не равно	SF<>OF
le ng	Если меньше или равно Если не больше	ZF=1 или SF<>OF
ne nz	Если не равно Если не ноль	ZF=0
no	Если нет переполнения	OF=0
np po	Если нет четности Если нечетное	PF=0
ns	Если нет знака	SF=0
p pe	Если есть четность Если четное	PF=1
o	Если есть переполнение	OF=1
s	Если есть знак	SF=1

14 Команды работы со строками.

Строка-источник находится по адресу DS:SI, а строка-приемник в ES:DI

- MOVSB, MOVSW, MOVSD — копирование одного байта/слова/двойного слова из DS:SI в ES:DI.
- CMPSB, CMPSW, CMPSD — сравнивает один байт/слово/двойное слово источника и приемника и устанавливает флаги аналогично команде CMP. При использовании “CMPS приемник, источник” ассемблер сам определяет, какую из трех форм выбрать.
- SCASB, SCASW, SCASD — сравнивает содержимое AL/AX/EAX с байтом/словом/двойным словом из памяти по адресу ES:DI и устанавливает флаги аналогично CMP. При использовании “SCAS приемник, источник” ассемблер сам определяет, какую из трех форм выбрать.
- LODSB, LODSW, LODSD — копирует байт/слово/двойное слово из памяти по адресу DS:SI. При использовании “LODS приемник, источник” ассемблер сам определяет, какую из трех форм выбрать.
- STOSB, STOSW, STOSD — копирует регистр AL/AX/EAX в память по адресу ES:DI.

Использование форм MOVS, CMPS, SCAS, LODS, STOS ассемблер сам определяет, какую из трех форм выбрать.

Разрешается заменить регистр DS другим с помощью префикса замены сегмента ES:, GS:, FS:, CS:, SS:. Регистр ES заменить нельзя.

В случае DF=0 после выполнения команды регистры SI, DI увеличиваются на 1, 2 или 4 (байт/слово/двойное слово).

Если DF=1, то уменьшаются на те же значения.

Все строковые команды работают только с одним элементом строки (байт, слово, двойное слово). Чтобы команда выполнялась

над всей строкой, необходим один из префиксов повторения операций:

- REP — повторять столько раз, сколько указано в регистре CX (при выполнении команды он уменьшается на 1)
- REPE — повторять, пока равно (завершение при ZF=0)
- REPNE — повторять, пока не равно (завершение при ZF=1)
- REPZ — повторять, пока ноль (завершение при ZF=0)
- REPNZ — повторять, пока не ноль (завершение при ZF=1)

Команды группы CMPS, SCAS в случаях REPNE, REPNZ сравнивают до первого совпадения, а в случаях REPE, REPZ сравнивают до первого несовпадения.

15 Команда трансляции по таблице.

Трансляция в соответствии с таблицей.

XLAT адрес

Помещает в AL байт из таблицы по адресу ES:BX со смещением относительно начала таблицы равным AL. Адрес, указанный в исходном коде, не обрабатывается компилятором и служит в качестве комментария. Если комментарий не нужен, можно написать XLATB.

Эта команда преобразует байт в соответствии с таблицей преобразований, поэтому часто используется для шифрования методом подстановки.

Можно написать следующий вариант преобразования* шестнадцатеричного числа в ASCII-код соответствующего ему символа:

```
mov     al, 0Ch
mov     bx, offset htable
xlatb
```

Если в сегменте данных, на которых указывает регистр ES, было записано

```
htable db "0123456789ABCDEF",
```

то теперь AL содержит не число 0Ch, а ASCII-код буквы "C".

* Но это преобразование можно сделать быстрее с помощью DAS.

16 Команда вычисления эффективного адреса.

Вычисление эффективного адреса

LEA приемник, источник

Вычисляет эффективный адрес (смещение?) источника (переменная) и помещает его в приемник (регистр). С помощью LEA можно вычислить адрес переменной, которая описана сложным методом адресации.

Команду LEA часто используют для быстрых арифметических вычислений, например, умножения, сложения:

*lea bx,[ebx+ebx*4] ;BX=EBX * 5*

lea ebx,[eax+12] ;EBX=EAX+12

Эти команды меньше, чем соответствующие MOV и ADD, и не изменяют флаги.

17 Структура программы на языке ассемблера.

Модули. Сегменты.

Описание строки программы

метка команда/директива операнды ; комментарий

Любое поле может быть опущено.

Если метка располагается перед командой процессора, сразу после нее всегда ставится оператор “:”, который указывает ассемблеру, что надо создать переменную с этим именем, содержащую адрес текущей команды.

Метки

В коде

- Пример:

```
mov cx, 5

label1:

add ax, bx

loop label1
```

- Метки обычно используются в командах передачи управления

В данных

- label
 - метка label тип
 - Возможные типы: BYTE, WORD, DWORD, FWORD, QWORD, TBYTE, NEAR, FAR.
- EQU, =
 - label EQU выражение
 - макрос
 - вычисляет выражение в правой части и приравнивает его метке

Любая программа состоит из одного или нескольких **сегментов** (блоков памяти размером до 64 КБ). Сегменты могут быть следующих типов: кода, данных, стека. За адрес начала сегмента отвечают сегментные регистры: для кода - CS, для стека - SS, для данных - DS и дополнительные ES, FS, HS.

Директивы SEGMENT (операнды необязательны), ENDS описывают сегмент программы.

```
имя SEGMENT [READONLY] выравнивание тип разряд 'класс'
...
имя ENDS
```

Имя сегмента — это метка, которая будет использоваться для получения сегментного адреса, а также для комбинирования сегментов в группы.

- READONLY — операнд, при котором MASM выдаст сообщение об ошибке на все команды, выполняющие запись в данный сегмент. Остальные ассемблеры игнорируют
- Выравнивание — указывает ассемблеру и компоновщику, с какого адреса может начинаться сегмента:
 - BYTE — с любого адреса
 - WORD — начало слова
 - DWORD — начало двойного слова
 - PARA — начало параграфа (по умолчанию)
 - PAGE — начало страницы (256 байт)
- Тип — возможный тип комбинирования сегментов:
 - PUBLIC — означает, что все такие сегменты с одинаковым именем, но разными классами будут объединены в один.
 - STACK — то же самое, что и public, но должен использоваться для сегментов стека.
 - COMMON — сегменты с одинаковым именем также объединяются в одни, но не последовательно, а по одному и тому же адресу, следовательно, длина суммарного сегмента будет равна длине максимального.
 - AT — указывает, что сегмент должен располагаться по фиксированному абсолютному адресу.
 - PRIVATE (по умолчанию) — сегмент такого типа не объединяется с другими сегментами
- Разрядность — имеет два значения USE16 и USE32. Первая используется по умолчанию (если перед .MODEL не применялась директива задания допустимого набора команд .386 и старше) и указывает, что сегмент не может превышать 64 Кб, а его команды и адреса является 16-разрядными. Сегменты USE32 могут занимать до 4 Гб, и все команды и адреса 32-битные.

- Класс сегмента — это любая метка, взятая в одинарные кавычки. Все сегменты с одинаковым классом, даже сегменты типа PRIVATE, будут расположены в исполняемом файле непосредственно друг за другом.

Как и на других языках программирования, программа на ассемблере может состоять из нескольких файлов — **модулей**. При компиляции (трансляции) каждый модуль превращается в объектный файл, далее при компоновке объектные файлы соединяются в единый исполняемый модуль.

END [точка_входа]

- точка_входа - имя метки, объявленной в сегменте кода и указывающее на команду, с которой начнется исполнение программы.
- Если в программе несколько модулей, только один может содержать точку входа.

18 Виды ассемблеров. Intel-синтаксис, AT&T-синтаксис.

Виды трансляторов ассемблера:

- MASM (Macro Assembler) — ассемблер от фирмы Microsoft. Сначала был коммерческим продуктом, затем начал поставляться в составе бесплатных (Visual C++). MASM является основным для DOS, Windows
- TASM (Turbo Assembler) — ассемблер фирмы Borland, более удобен для 16-битных программ для DOS
- NASM (Netwide Assembler) — бесплатный ассемблер с открытым исходным кодом под лицензией GNU Lesser General Public License. Кроссплатформенный, т.е. подходит под DOS, Windows, Unix и т.д.
- FASM (Flat Assembler) — кроссплатформенный ассемблер с открытым исходным кодом под лицензией GNU Public License. Имеет поддержку ELF-файлов
- as — стандартный ассемблер практически во всех разновидностях Unix, в том числе Linux, BSD. Свободная версия называется gas (GNU assembler). Синтаксис сильно отличается от предыдущих

Основны ассемблеры для MS DOS и Windows используют синтаксис, предложенный компанией Intel. Ассемблер для Unix (as, gas) кардинально отличается от него.

Основные отличия AT&T-синтаксиса от Intel-синтаксиса:

1. Имена регистров предваряются префиксом %.
2. Обратный порядок операндов: вначале источник, затем приёмник.
3. Размер операнда задается суффиксом, замыкающим инструкцию.
4. Числовые константы записываются в Си-соглашении.
5. Для получения смещения метки используется префикс \$.

19 Подпрограммы. Объявление, вызов.

Ассемблер не накладывает на подпрограммы никаких ограничений – на любой адрес программы можно передать управление командой CALL, и оно вернется к вызвавшей программе, как только встретится команда RET.

Директива объявления подпрограммы (для MASM / WASM):

```
метка  proc      тип          язык      USES регистры
      ...
      ret
метка  endp
```

Все операнды proc необязательны.

- Тип может принимать значения NEAR, FAR. По умолчанию в моделях памяти TINY, SMALL, COMPACT подпрограмма имеет тип NEAR.
- Язык может принимать значения C, PASCAL, BASIC, FORTRAN, SYSCALL, STDCALL. Если он указан, подразумевается, что подпрограмма рассчитана на вызов из программ на соответствующем ЯВУ.
- USES – список регистров, значения которых изменяет процедура.

CALL *операнд*

Операндом может быть непосредственное значение адреса (метка в ассемблерных программах), регистр или переменная, содержащие адрес перехода. Если в качестве адреса перехода указано только смещение, считается, что адрес расположен в том же сегменте, что и команда CALL. Процессор помещает значение регистра IP, соответствующее следующей за CALL команде, в стек и загружает в IP новое значение, осуществляя тем самым передачу управления. При дальнейшем переходе в стек также помещается значение регистра CS.

20 Подпрограммы. Возврат управления.

Ассемблер не накладывает на подпрограммы никаких ограничений – на любой адрес программы можно передать управление командой CALL, и оно вернется к вызвавшей программе, как только встретится команда RET.

Директива объявления подпрограммы (для MASM / WASM):

```
метка  proc      тип          язык      USES регистры
      ...
      ret
метка  endp
```

Все операнды proc необязательны.

- Тип может принимать значения NEAR, FAR. По умолчанию в моделях памяти TINY, SMALL, COMPACT подпрограмма имеет тип NEAR.
- Язык может принимать значения C, PASCAL, BASIC, FORTRAN, SYSCALL, STDCALL. Если он указан, подразумевается, что подпрограмма рассчитана на вызов из программ на соответствующем ЯВУ.
- USES – список регистров, значения которых изменяет процедура.

RETN / RETF / RET число

RETN считывает из стека слово (или двойное слово) и загружает его в IP, передавая управление обратно программе, из которой вызывалась команда CALL.

RETF загружает из стека IP и CS, возвращаясь из дальней подпрограммы.

В случае RET ассемблер сам заменит ее на RETN или RETF.

Число необязательно, но если оно указано, то из стека после считывания адреса возврата будет удалено указанное число байт – это нужно, если при вызове процедуры ей передавались параметры через стек.

21 Макроопределения.

Макроопределение (макрос) — именованный участок программы, который ассемблируется каждый раз, когда его имя встречается в тексте программы.

Макрос начинается директивой `MACRO` и заканчивается директивой `ENDM`

Пример:

```
load_reg MACRO register1, register2  
    push register1  
    pop register2  
ENDM
```

В программе можно использовать слово `load_reg`, как если бы это было имя команды, и ассемблер заменит каждое такое слово на две команды, содержащиеся в макроопределении. Можно было бы оформить этот же участок кода в виде подпрограммы, и если эта подпрограмма вызывается больше одного раза, этот вариант программы займет меньше места, но вариант с

макроопределением станет выполняться быстрее, так как нет лишних команд `CALL` и `RET`.

Еще одним плюсом макросов является **возможность вызова с параметрами**.

Макрооперации:

- `%` — находящийся дальше текст является выражением и должен быть сначала вычислен
- `<>` — подстановка текста без изменений (передача текстовых строк, списка параметров вложенному макросу)
- `&` — параметр переданный в качестве операнда макросу, заменяется значением до обработки строки ассемблером
- `!` — считать следующий символ текстом, а не знаком операции
- `;;` — начало макрокомментария

22 стек. Аппаратная поддержка вызова подпрограмм.

Стек

- LIFO/FILO (last in, first out) - последним пришёл, первым ушёл
- Сегмент стека - область памяти программы, используемая её подпрограммами, а также (вынужденно) обработчиками прерываний
- SP - указатель на вершину стека
- В x86 стек "растет вниз", в сторону уменьшения адресов. При запуске программы SP указывает на конец сегмента

Команды непосредственной работы со стеком:

- PUSH <источник> - поместить данные в стек. Уменьшает SP на размер источника и записывает значение по адресу SS:SP.
- POP <приемник> - считать данные из стека. Считывает значение с адреса SS:SP и увеличивает SP.
- PUSHAD - поместить в стек регистры AX, CX, DX, BX, SP, BP, SI, DI.
- POPAD - загрузить регистры из стека (SP игнорируется)
- PUSHF - поместить в стек содержимое регистра флагов
- POPF - загрузить регистр флагов из стека

Стековый кадр (фрейм) — механизм передачи аргументов и выделения временной памяти с использованием аппаратного стека. Содержит информацию о состоянии подпрограммы.

Включает в себя:

- параметры
- адрес возврата (обязательно)
- локальные переменные

23 Соглашения о вызовах. Понятие, основные виды соглашений.

Соглашения о вызовах (calling conventions) — описания технических особенностей вызова подпрограмм, определяющие:

- способ передачи параметров подпрограммам;
- способ передачи управления подпрограммам;
- способ передачи результатов выполнения из подпрограмм в точку вызова;
- способ возврата управления из подпрограмм в точку вызова.

Распространённые соглашения:

- cdecl
- pascal
- stdcall (Win32 API)
- fastcall
- thiscall - вызов нестатических методов C++ (this через ECX)
- Microsoft x64

CDECL

Соглашение о вызовах по умолчанию для 32-битных программ C и C++. Отличительные особенности:

- все аргументы передаются через стек, справа налево (тем самым, первый аргумент будет ближе к SP, а последний — дальше)
- вызывающая функция занимается очищением стека, а не вызванная

System V (linux x64)

Соглашение о вызовах, использовавшееся в System V Unix, а сейчас применяющееся в компиляторе gcc под Unix-like операционные системы (например, linux)

Отличительные особенности:

- Первые 6 аргументов, относящихся к целочисленному типу или указателю, передаются через регистры rdi, rsi, rdx, rcx, r8, r9. Остальные аргументы передаются через стек в соответствии с cdecl.
- Первые 8 аргументов, относящихся к числам с плавающей запятой, передаются через регистры XMM (расширения SSE), остальные через стек как в первом случае

24 Прерывания. Обработка прерываний в реальном режиме работы процессора.

Прерывание — особая ситуация, когда выполнение текущей программы приостанавливается и управление передаётся программе-обработчику возникшего прерывания.

Виды прерываний:

- аппаратные (асинхронные) - события от внешних устройств;
- внутренние (синхронные) - события в самом процессоре, например, деление на ноль;
- программные - вызванные командой INT.

Для приема **аппаратных (внешних) прерываний** у процессора есть два физических контакта — NMI (Non-Maskable Interrupt) и INTR(Interrupt Request). В зависимости от того, на какой вход передается сигнал аппаратный прерывания делятся на:

- **маскируемые (INTR)** — те, которые можно запретить, т.е. программист может указать процессору, что прерывания этого типа не надо обслуживать, связаны с нормальной работой внешних устройств (клавиатура, мышь, жесткий диск, сопроцессор и т.д.)
- **немаскируемые (NMI)** — запретить нельзя, они должны быть обслужены сразу, возникают в результате чрезвычайной ситуации (сбой в памяти, временное понижение напряжения)

При возникновении прерывания процессор прекращает выполнение обрабатываемой программы и переключается на процедуру обработки прерывания. Перед этим процессор сохраняет в стек значения CS, IP и FLAGS. После обработки прерывания прерванная программа продолжает свое выполнение.

Обработка прерываний в реальном режиме:

В самом начале памяти, начиная с 0, расположена таблица 256 векторов прерываний. Вектор прерывания — идентифицирует соответствующий его номеру обработчик прерывания, занимает 4Б (полный адрес). Вся таблица занимает 1 Кб.

Перехват прерываний:

При написании обработчика в конце необходимо добавить команду **IRET**, которая восстанавливает флаги FLAGS, CS:IP, выходит из обработчика.

- Сохранение адреса старого обработчика
- Изменение вектора на "свой" адрес
- Вызов старого обработчика до/после отработки своего кода
- При деактивации - восстановление адреса старого обработчика

Привязка к выбранному номеру прерывания осуществляется с помощью прерываний:

- int 21h, AH=35h, AL=номер прерывания — возвращает в ES:BX адрес обработчик
- int 21h, AH=25h, AL=номер прерывания — устанавливает адрес нового обработчика из DS:DX в таблице прерываний

Некоторые прерывания:

- 0 - деление на 0
- 1 - прерывание отладчика, вызывается после каждой команды при флаге TF
- 3 - "отладочное", int 3 занимает 1 байт
- 4 - переполнение при команде INTO (команда проверки переполнения)
- 5 - при невыполнении условия в команде BOUND (команда контроля индексов массива)
- 6 - недопустимая (несуществующая) инструкция

- 7 - отсутствует FPU
- 8 - таймер
- 9 - клавиатура
- 10h - прерывание BIOS

Прерывание BIOS 10h

АН = 00h	установка видеорежима, код в AL
АН = 02h	установить позицию курсора
АН = 08h	считать символ и атрибуты из позиции курсора
АН = 09h	записать символ и атрибуты в позицию курсора
АН = 0Ch	задать пиксель
АН = 0Dh	прочитать цвет пикселя

25 Процессор 80386. Режимы работы. Регистры.

Производство x86 (386+): 1985 – ~2010

32-разрядные:

- Регистры, кроме сегментных
- Шина данных
- Шина адреса ($2^{32} = 4\text{Гб ОЗУ}$)

Режимы работы

8086 (1978 г.) -> 80186 (1982 г.)

-> 80286 (1982 г.) добавлен защищённый режим

-> 80386 (1985 г.) архитектура стала 32-разрядной

-> 80486 (1989 г.) -> Pentium -> ... -> (современные процессоры)

"Реальный" режим (режим совместимости с 8086)

- обращение к оперативной памяти происходит по реальным (действительным) адресам, трансляция адресов не используется;
- набор доступных операций не ограничен;
- защита памяти не используется.

"Защищённый" режим

- обращение к памяти происходит по виртуальным адресам с использованием механизмов защиты памяти;
- набор доступных операций определяется уровнем привилегий (кольца защиты): системный и пользовательский уровни

Режим V86, ...

Система команд

- Аналогична системе команд 16-разрядных процессоров
- Доступны как прежние команды обработки 8- и 16-разрядных аргументов, так и 32-разрядных регистров и переменных
- Пример:

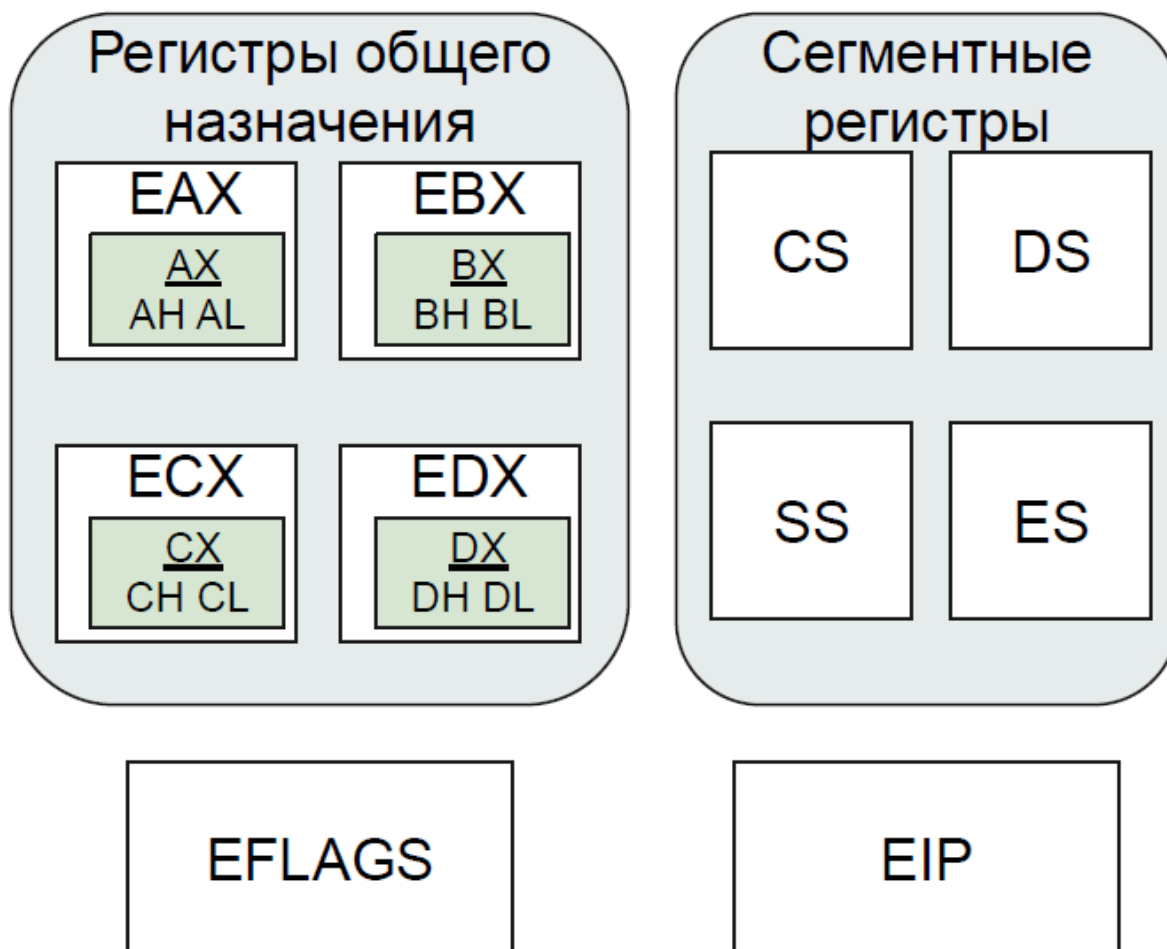
```
mov eax, 12345678h
```

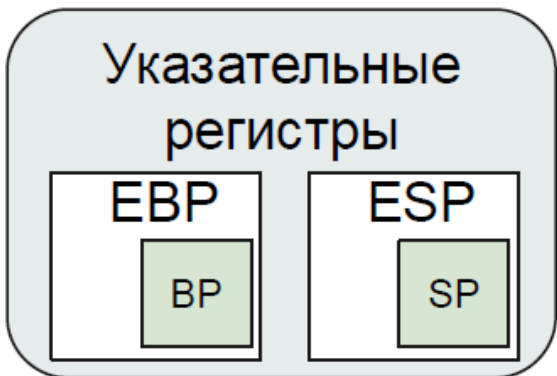
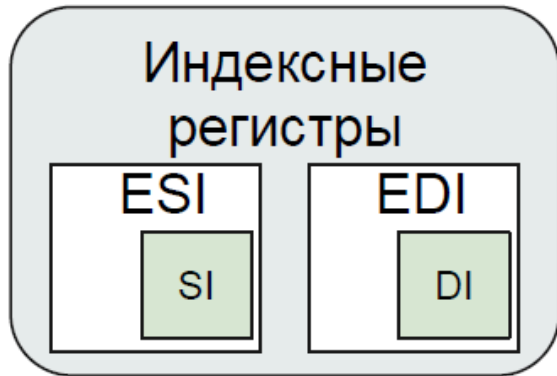
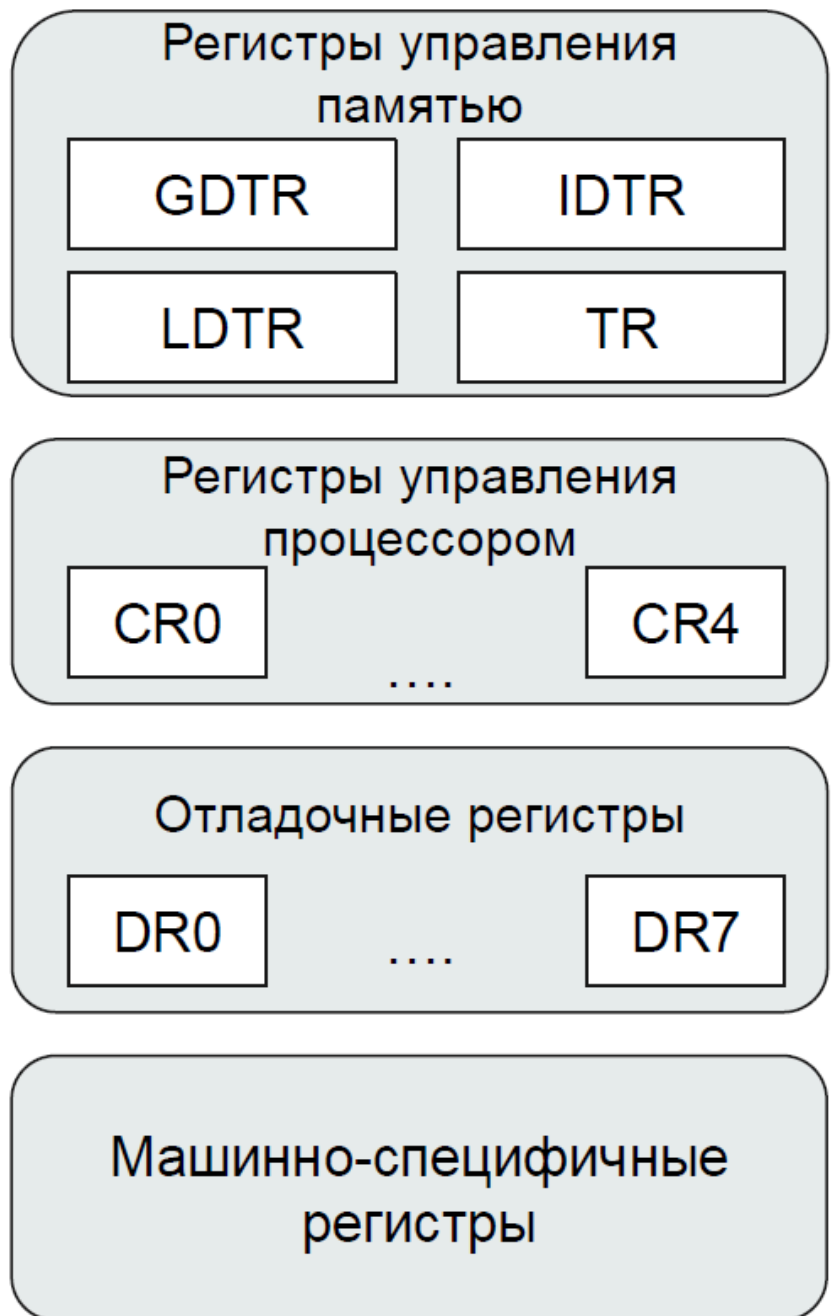
```
xor ebx, ebx
```

```
mov bx, 1
```

```
add eax, ebx ; eax=12345679h
```

Регистры x86





В EFLAGS добавляется к FLAGS еще 5 специфичных флагов.

Регистры управления памятью:

- GDTR: 6-байтный регистр, содержит 32-битный линейный адрес начала таблицы глобальных дескрипторов (GDT) и 16-битный размер (лимит, уменьшенный на 1)
- IDTR: 6-байтный регистр, содержит 32-битный линейный адрес начала таблицы глобальных дескрипторов обработчиков прерываний (IDT) и 16-битный размер (лимит, уменьшенный на 1)
- LDTR: 10-байтный регистр, содержит 16-битный селектор для GDT и весь 8-байтный дескриптор из GDT, описывающий текущую таблицу локальных дескрипторов

- TR: 10-байтный регистр, содержит 16-битный селектор для GDT и весь 8-байтный дескриптор из GDT, описывающий TSS текущей задачи

Регистры управления процессором:

- CR0 - флаги управления системой
 - PG - включение режима страничной адресации
 - управление отдельными параметрами кеша
 - WP - запрет записи в страницы "только для чтения"
 - NE - ошибки FPU вызывают исключение, а не IRQ13
 - TS - устанавливается процессором после переключения задачи
 - PE - включение защищенного режима
- CR1 - зарезервирован
- CR2 - регистр адреса ошибки страницы - содержит линейный адрес страницы, при обращении к которой произошло исключение #PF
- CR3 - регистр основной таблицы страниц
 - 20 старших бит физического адреса начала каталога таблиц либо 27 старших бит физического адреса начала таблицы указателей на каталоги страниц, в зависимости от бита PAE в CR4
 - Управление кешированием и сквозной записью страниц
- CR4 - регистр управления новыми возможностями процессоров (с Pentium)

Отладочные регистры:

- DR0..DR3 - 32-битные линейные адреса четырёх возможных точек останова по доступу к памяти
- DR4, DR5 - зарезервированы
- DR6 (DSR) - регистр состояния отладки. Содержит причину останова

- DR7 (DCR) - регистр управления отладкой. Управляет четырьмя точками останова

Машинно-специфичные регистры

- Управление кешем
- Дополнительное управление страничной адресацией
- Регистры расширений процессора: MMX и т.д.

26 Страничная модель памяти. Виртуальная память.

Страничная модель — виртуальные адреса отображаются на физические постранично:

- **виртуальная память** — метод управления памятью компьютера, позволяющий выполнять программы, требующие больше оперативной памяти, чем имеется в компьютере, путем автоматического перемещения частей программы между основной памятью и вторичным хранилищем (файл, или раздел подкачки)
- основной режим для большинства современных ОС
- в x86 минимальный размер страницы - 4096 байт
- основывается на таблице страниц - структуре данных, используемой системой виртуальной памяти в операционной системе компьютера для хранения сопоставления между виртуальным адресом и физическим адресом. Виртуальные адреса используются выполняющимся процессом, в то время как физические адреса используются аппаратным обеспечением. Таблица страниц является ключевым компонентом преобразования виртуальных адресов, который необходим для доступа к данным в памяти.

Страничная организация памяти



Управление памятью в x86:

- В сегментных регистрах - селекторы:
 - 13-разрядный номер дескриптора
 - какую таблицу использовать - глобальную или локальную
 - уровень привилегий запроса 0-3
- По селектору определяется запись в одной из таблиц дескрипторов сегментов
- При включенном страничном режиме - по таблице страниц определяется физический адрес страницы либо выявляется, что она выгружена из памяти, срабатывает исключение и операционная система подгружает затребованную страницу из "подкачки" (swap)

Поддержка многозадачности

TSS (Task State Segment — сегмент состояния задачи) — специальная структура в архитектуре x86, содержащая информацию о задаче (процессе). Используется ОС для диспетчеризации задач, в т. ч. переключения на стек ядра при обработке прерываний и исключений

27 Математический сопроцессор. Типы данных.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Одним из таких расширений является **математический сопроцессор**, называемый FPU – *Floating Point Unit*. Изначально был создан как отдельное опциональное устройство на материнской плате, но начиная с процессора Intel 80486 встраивается в процессор.

Математический сопроцессор создан для работы с числами с плавающей запятой (формой представления дробных чисел).

Число с плавающей запятой в FPU представляется как произведение $x = 1, n_1 n_2 \dots \cdot 2^p$, где p называется экспонентой числа x , а $1, n_1 n_2 \dots$ – мантиссой числа x . Знак мантиссы хранится отдельным битом, сама мантисса хранится без первой цифры, так как это всегда 1.

В сопроцессоре определены операции над следующими типами данных:

- целое слово (16 бит)
- короткое слово (32 бит)
- длинное слово (64 бит)
- упакованное десятичное число (80 бит)
- короткое вещественное число (float 32 бит)
- длинное вещественное число (double 64 бит)
- расширенное вещественное число (80 бит)

Также в FPU имеются особые числа:

- положительная бесконечность (знак 0, мантисса 0, экспонента единицы)
- отрицательная бесконечность (знак 1, мантисса 0, экспонента единицы)
- денормализованные числа (экспонента = 0): поскольку на экспонента является положительным числом (экспонента есть

|минимальная степень| + фактическая степень + 1, например:
если экспонента ограничена 4 битами, то минимальная
степень равна -8, то для числа $0.625 = 1,01b \cdot 2^{-1}$
экспонентой будет являться не -1, а $8 - 1 + 1 = 8$), то нулевая
указывает на то, что число является меньше любого другого
числа, представимого в виде нормализованного числа с
плавающей запятой

28 Математический сопроцессор. Регистры.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Одним из таких расширений является **математический сопроцессор**, называемый FPU – *Floating Point Unit*. Изначально был создан как отдельное опциональное устройство на материнской плате, но начиная с процессора Intel 80486 встраивается в процессор.

Математический сопроцессор создан для работы с числами с плавающей запятой (формой представления дробных чисел).

Число с плавающей запятой в FPU представляется как произведение $x = 1, n_1 n_2 \dots \cdot 2^p$, где p называется экспонентой числа x , $1, n_1 n_2 \dots$ – мантиссой числа x . Знак мантиссы хранится отдельным битом, сама мантисса хранится без первой цифры, так как это всегда 1.

Регистры

Расширение FPU включает в себя 8 новых 80-разрядных регистров R0..R7. Данные регистры называются регистрами данных сопроцессора, а доступ к ним реализован через имена STn, организованных в виде стека: ST соответствует вершине стека, ST(1)..ST(7) – следующие за вершиной регистры.

В дополнение к 8 регистрам данных данное расширение добавляет вспомогательные регистры:

- SR – регистр состояний (2 байта), содержит слово состояния FPU. Сигнализирует о различных ошибках, переполнениях. Биты 13,12,11 (число от 0 до 7) говорят о том, на какой регистр данных Rn указывает имя ST (какой регистр данных является вершиной стека)
- CR – регистр управления (2 байта). Контролирует правила округления, точности.

- TW – регистр тэгов (2 байта). 8 пар битов, каждая пара соответствует своему регистру данных FPU и определяет состояние регистра: число, ноль, NaN, пусто
- FIP и FDP (48 бит каждый) – содержат адрес последней выполненной команды и ее операнда для обработки исключений

29 Математический сопроцессор. Классификация команд.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Одним из таких расширений является **математический сопроцессор**, называемый FPU – *Floating Point Unit*. Изначально был создан как отдельное опциональное устройство на материнской плате, но начиная с процессора Intel 80486 встраивается в процессор.

Математический сопроцессор создан для работы с числами с плавающей запятой (формой представления дробных чисел). **Число с плавающей запятой** в FPU представляется как произведение $x = 1, n_1 n_2 \dots \cdot 2^p$, где p называется экспонентой числа x , $1, n_1 n_2 \dots$ – мантиссой числа x . Знак мантиссы хранится отдельным битом, сама мантисса хранится без первой цифры, так как это всегда 1.

Классификация команд:

- **Команды пересылки:**

- **FLD источник** — команда помещает содержимое источника в стек и уменьшает TOP на 1
- **FST приемник** — копирует ST(0) в приемник
FSTP — FST + POP, увеличивает TOP на 1
- **FILD, FIST, FISTP** — работа с целыми числами (внутри стека они вещественные, преобразуются на входе/выходе), аналоги перечисленных выше команд для целых чисел
- **FBLD, FBSTP** — работа с двоично-десятичными BCD числами, аналоги перечисленных выше команд
- **FXCH источник** — меняет местами содержимое ST(0) и ST(n), где n – источник, по умолчанию $n=1$
- **FCMOVcc приемник, источник** — копирует источник (ST(n)) в приемник (только ST(0)) при выполнении условия

● Базовая арифметика:

- **FADD** *приемник, источник* — сложение источника и приемника помещается в приемник.
FADDP *приемник, источник* — **FADD + POP**
FIADD *источник* — сложение источника (целое число) и ST(0) записывается в ST(0)
- **FSUB** *приемник, источник* — вычитание источника из приемника записывается в приемник
FSUBP *приемник, источник* — **FSUB + POP**
FISUB *источник* — источником является целое число, приемником ST(0)
- **FSUBR, FSUBP, FISUBR** — аналогично, но вычитание приемника из источника, запись в приемник
- **FMUL** *приемник, источник* — умножение источника на приемник, запись в приемник
FMULP *приемник, источник* — **FMUL + POP**
FIMUL *источник* — умножение источника на ST(0)
- **FDIV** *приемник, источник* — деление приемника на источник, запись в приемник
FDIVP *приемник, источник* — **FDIV + POP**
FIDIV *источник* — деление ST(0) на источника
- **FDIVR, FDIVRP, FIDIVR** — аналогично, но деление источника на приемник, запись в приемник
- **FPREM** — деление ST(0) на ST(1), остаток от деления в ST(0), деление осуществляется последовательным вычитанием (не более 64 раз)
- **FABS** — переводит ST(0) в его абсолютное значение
- **FCHS** — меняет знак ST(0) на противоположный
- **FRNDINT** — округляет ST(0) до целого в соответствии с режимом округления, заданным в CR
- **FSCALE** — умножает ST(0) на 2 в степени ST(1) и записывает в ST(0), значение ST(1) предварительно округляется в сторону нуля до целого числа

- **FXTRACT** — разделяет число в ST(0) на мантиссу и экспоненту, сохраняет экспоненту в ST(0) и помещает мантиссу в стек, так что после этого TOP уменьшается на 1, мантисса оказывается в ST(0), а экспонента в ST(1)
- **FSQRT** — извлекает квадратный корень

● Команды сравнения:

- **FCOM *источник*** — сравнивает ST(0) с источником ST(n), устанавливает флаги
FCOMP *источник* — FCOM + POP
FCOMPP — сравнивает ST(0) и ST(1), выталкивает из стека оба числа
- **FUCOM, FUCOMP, FUCOMPP** — аналогичны, разница в работе с QNAN (все флаги в 1, но исключение “недопустимая операция” не вызывается)
- **FICOM, FICOMP *источник*** — аналогично, но с целыми числами
- **FCOMI, FCOMIP, FUCOMI, FUCOMIP *источник*** — аналогично FCOM, FCOMP, FUCOM, FUCOMP, но устанавливает флаги не SR, а флаги EFLAGS
- **FTST** — проверяет, есть ли ноль в ST(0), флаги в SR
- **FXAM** — анализирует ST(0) на тип числа (неподдерживаемое, не-число, нормальное конечное, бесконечность, ноль, пустой регистр, денормализованное)

● Трансцендентные операции:

- **FSIN** — синус ST(0), операнд от -2^{63} до 2^{63}
- **FCOS** — косинус ST(0), операнд от -2^{63} до 2^{63}
- **FSINCOS** — синус ST(0) помещает в ST(1), а косинус в ST(0)
- **FPTAN** — тангенс ST(0) помещается в ST(1), а в ST(0) помещается 1, чтобы можно было вычислить котангенс с помощью FDIVR
- **FPATAN** — арктангенс числа, получаемого при делении ST(1), результат в ST(0), смысл в вычислении угла между

осью абсцисс и линией проходящей через центр и точку $ST(1)$, $ST(0)$

- **F2XM1** — возводит 2 в степень $ST(0)$ (от -1 до +1) и вычитает 1, результат в $ST(0)$
- **FYL2X** — вычисляет $ST(1)$, умноженное на $\log_2(ST(0))$, помещает в $ST(0)$, $ST(0)$ неотрицательное
- **FYL2XP1** — вычисляет $ST(1)$, умноженное на $\log_2(ST(0)+1)$, $ST(0)$ должно быть от $-(1-(\sqrt{2})/2)$ до $(1+(\sqrt{2})/2)$.

30 Расширения процессора. MMX. Регистры, поддерживаемые типы данных.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении MMX. Расшифровывается как Multimedia Extensions. Одно из первых расширений x86, реализующее SIMD для увеличения эффективности обработки больших потоков данных.

Расширение добавляет 8 64-битных регистров MM0-MM7 – мантиссы регистров FPU. При использовании MMX пользоваться регистрами FPU не получится, при этом при записи в MMn

экспонента и знаковый бит зануляются

Все регистры MMX поддерживают следующие типы данных:

- одно четверное слово (64-битное число)
- два упакованных двойных слова
- четыре упакованных слова
- восемь упакованных байта

31 Расширения процессора. MMX. Классификация команд.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

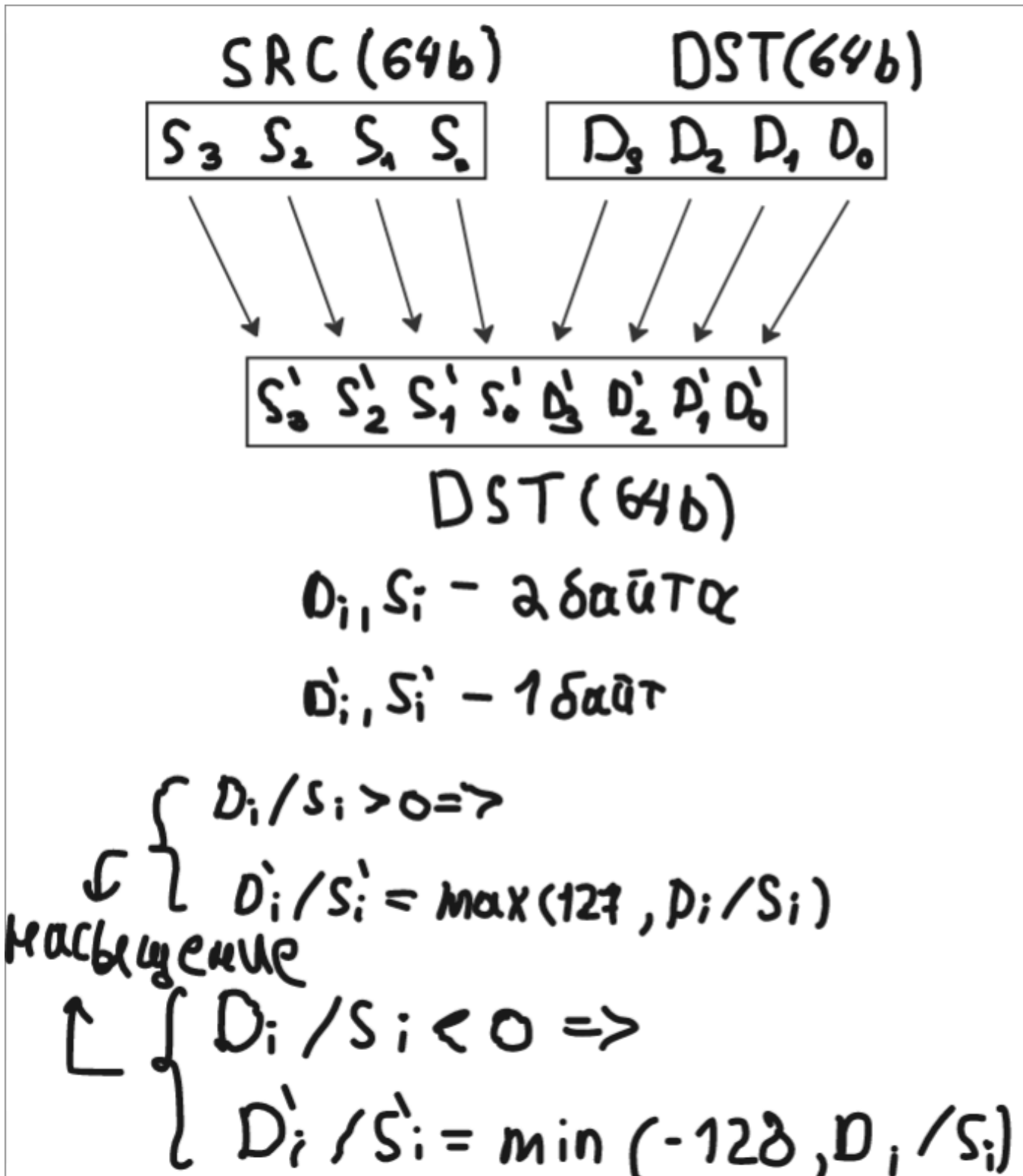
Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении MMX. Расшифровывается как Multimedia Extensions. Одно из первых расширений x86, реализующее SIMD для увеличения эффективности обработки больших потоков данных.

Основные команды расширения MMX:

- Команды пересылки данных:
 - MOVD, MOVQ – пересылка двойных/четверных слов;
 - PACKSSWB, PACKSSDW – упаковка слов в байты или двойных слов в слова с использованием знакового насыщения



- `PACKUSWB` – упаковка слов в байты с беззнаковым насыщением;

- Арифметические операции
 - PADDDB, PADDW, PADDW – поэлементное сложение, без переноса
 - PADDSB, PADDSW – сложение с насыщением
 - PADDUSB, PADDSW – беззнаковое сложение с насыщением
 - PMULHW, PMULLW - старшее/младшее умножение (сохраняет старшую или младшую части результата в приёмник)
 - PMADDWD - умножение и сложение. Перемножает 4 слова, затем попарно складывает произведения двух старших и двух младших
- Команды сравнения
 - PCMPEQB, PCMPEQW, PCMPEQD - проверка на равенство. Если пара равна – соответствующий элемент приемника заполняется единицами, иначе - нулями
 - PCMPGTB, PCMPGTW, PCMPGTD - сравнение. Если элемент приемника больше, то заполняется единицами, иначе - нулями
- Логические операции
 - PAND, POR – логическое И/ИЛИ
 - PANDN – логическое НЕ-И
 - PXOR – исключающее или
- Сдвиговые операции
 - PSLLW, PSLLD, PSLLQ – логический сдвиг влево
 - PSRLW, PSRLD, PSRLQ – логический сдвиг вправо
 - PSRAW, PSRAD – арифметический сдвиг вправо
 - Все перечисленные выше команды имеют два аргумента: приемник (где сдвинуть) и сдвиг (на сколько бит сдвинуть), приемник и сдвиг являются регистрами mmx

32 Расширения процессора. SSE. Регистры, поддерживаемые типы данных.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении SSE. Расшифровывается как Streaming SIMD Extensions, создан для поддержки SIMD инструкций, призванных решить проблему параллельной работы с FPU. Первая версия была представлена в 1999 году.

Первая версия поддерживает 8 128-разрядных регистров XMM0-XMM7, которые поддерживают два типа данных: упакованный вектор из 4 чисел с плавающей запятой одинарной

точности и одно упакованное число одинарной точности

Вторая версия (SSE2) стала поддерживать два новых типа данных для регистров XMM в дополнение к двум уже имеющимся: упакованный вектор из 2 чисел с плавающей запятой двойной точности (по 64 бит) и упакованное число двойной точности.

33 Расширения процессора. SSE. Классификация команд.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении SSE. Расшифровывается как Streaming SIMD Extensions, создан для поддержки SIMD инструкций, призванных решить проблему параллельной работы с FPU. Первая версия была представлена в 1999 году.

Вторая версия SSE2 начала поддерживать числа двойной точности (1 неупакованное или 2 упакованных числа), является окончательной заменой расширения MMX

Основные особенности:

- добавляет 8 128-разрядных регистров (16 в x64) XMM
- добавляет свой регистр флагов для операций над числами с плавающей запятой
- основной тип данных – вещественные числа одинарной точности (1 неупакованное или 4 упакованных)
- поддержка регистров MMX для целочисленных команд

Расширение SSE включает в себя команды для работы с регистрами XMM:

- пересылки данных (MOVSS, MOVUPS и другие);
- арифметические (ADDSS, ADDPS и другие);
- сравнения (CMPSS, CMPPS и другие);
- преобразования типов (CVTSS2SS, CVTSS2PS и другие);
- логические (ANDPS, ORPS и другие);
- целочисленные
- упаковки

34 Расширения процессора. AVX. Регистры, поддерживаемые типы данных.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении AVX. Расшифровывается как *Advanced Vector Extensions*, создан для поддержки 256-битных SIMD инструкций, продолжение расширений SSE1/2/3/4. Первая версия была представлена в 2008 году.

Основные особенности:

- регистры увеличены со 128 (XMM) до 256 (YMM0-YMM15) бит;
- “неразрушающие” (трёхоперандные) инструкции: $c = a + b$

вместо $a = a + b$;

- SSE-инструкции используют младшую половину YMM-регистров, не меняя старшую часть;

Как было упомянуто выше, данное расширение поддерживает XMM0-XMM15 регистры расширения SSE для обратной совместимости, которые являются младшей частью больших регистров YMM0-YMM15, являющихся уже 256-битными.

Представленное в 2013 году улучшение AVX512 добавило также 512-битные регистры ZMM, частью которых являются регистры YMM, а число регистров всех размеров было увеличено до 32 (ZMM0-ZMM31).

Для регистров XMM предусмотрено 4 поддерживаемых типа данных: 2 упакованных 64 битных числа, 4 упакованных 32 битных числа, 1 неупакованное 32 или 64 битное число.

Для регистров YMM предусмотрено также 4 различных типа данных, однако в случае упакованных хранится 4 64-битных или 8 32-битных числа

Для регистров ZMM число хранимых чисел в случае упакованной структуры вновь умножается в 2 раза по сравнению с YMM

35 Расширения процессора. AVX. Классификация команд.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении AVX. Расшифровывается как *Advanced Vector Extensions*, создан для поддержки 256-битных SIMD инструкций, продолжение расширений SSE1/2/3/4. Первая версия была представлена в 2008 году.

Основные особенности:

- регистры увеличены со 128 (XMM) до 256 (YMM0-YMM15) бит;
- “неразрушающие” (трёхоперандные) инструкции: $c = a + b$

вместо $a = a + b$;

- SSE-инструкции используют младшую половину YMM-регистров, не меняя старшую часть;

Включает в себя команды, аналогичные командам SSE, однако продолженные на большую размерность регистров:

- пересылки данных (MOVSS, MOVUPS и другие);
- арифметические (ADDSS, ADDPS и другие);
- сравнения (CMPSS, CMPPS и другие);
- преобразования типов (CVTSI2SS, CVTPI2PS и другие);
- логические (ANDPS, ORPS и другие);
- целочисленные
- упаковки

36 Процессоры семейства x86-64. Регистры, режимы работы.

С 2001 года компания AMD выпускает процессоры на новой архитектуре AMD x64, главным образом отличающейся от старой архитектуры наличием 64-разрядными регистрами и инструкциями для работы с ними. С 2003 года Intel также начинает выпускать 64-разрядные процессоры. Процессоры этого семейства называют x86-64

Режимы работы процессоров x86-64:

- Legacy mode – режим совместимости с 32-разрядными процессорами
- Long mode – 64-разрядных режим с частичной поддержкой 32-разрядных программ. Не поддерживается V86 и сегментная модель памяти

Регистры процессоров x86-64:

- имеющиеся в предыдущих процессорах 32-битные регистры расширены до 64-битных, а именно: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP
- добавлено 8 новых целочисленных регистров общего назначения Rn: R8-R15, каждый по 64 бит
- указатель расширен до 64 бит – RIP, регистр флагов также расширен до 64 бит – RFLAGS

При этом регистры 32-разрядных процессоров, расширенных до 64-разрядных, доступны по тем же именам (EAX, ECX и так далее). Для новых регистров Rn:

- младшие 32 бит Rn доступны по имени RnD
- младшие 16 бит Rn доступны по имени RnW
- младшие 8 бит Rn доступны по имени RnB

37 Расширения процессора. AES. Назначение, классификация команд.

Расширением процессора называется аппаратный модуль, расширяющий возможности процессора. Для процессоров x86 существует множество различных расширений (о них ниже). Для просмотра имеющихся расширений процессора существует специальная команда CPUID (начиная с 80486 процессора), которая сообщает о имеющихся в наличии расширениях процессора.

Рассмотрим самые популярные расширения процессоров x86:

- FPU – математический сопроцессор, изначально располагающийся на материнской плате (с 80486 встроен в процессор). Необходим для операций над числами с плавающей запятой;
- MMX – реализует SIMD (64 бит регистры) для целых чисел;
- SSE – реализует SIMD (128 бит регистры) для чисел с плавающей запятой, развитие MMX, 1999 год;
- SSE2 – развитие SSE, 2000 год;
- SSE3 – развитие SSE2, 2004 год;
- AVX – развитие SSE4, поддерживает 256 битные регистры для вычисления операций с плавающей запятой;
- AES – для аппаратной поддержки шифрования по алгоритму AES

Подробнее остановимся на расширении AES. Расшифровывается как *Advanced Encryption Standard*, целью расширения является ускорение шифрования по алгоритму AES – стандартный алгоритм шифрования, принятый правительством США.

Команды, добавленные в данном расширении можно разделить на

три группы:

- команды раундов шифрования – AESENC и AESENCLAST;
- команды раундов расшифрования – AESDEC и AESDECLAST;
- команда способствования генерации ключа – AESKEYGENASSIST;

38 Архитектура RISC. Семейство процессоров ARM.

Версии архитектуры, профили.

Ранние архитектуры процессоров (комплексные, CISC (Complex instruction set computer)):

- большее количество команд;
- разные способы адресации для упрощения написания программ на ассемблере;
- поддержка конструкций языков высокого уровня.

Недостатки: на практике многие возможности CISC используются компиляторами ЯВУ ограничено, а их поддержка затратна.

Преимущества: более простой код ассемблера (из за большего количества команд, поддерживающих некоторые возможности языков высокого уровня)

Однако, с приходом ЯВУ и различных компиляторов, реализующих возможности команд CISC процессоров с использованием базовых инструкций, после чего появился RISC.

RISC (reduced instruction set computer) – архитектура, отличительными чертами которой от CISC являются:

- сведение набора команд к простым типовым;
- большее количество регистров (возможно, за счет общего упрощения архитектуры);
- стандартизация формата команд, упрощение конвейеризации.

ARM – семейство процессоров, альтернатива семейству x86-64 и семейству процессоров Intel x86. Расшифровывается как Advanced RISC Machine (усовершенствованная RISC машина)

Используется ARM главным образом в мобильных устройствах (процессоры Qualcomm, MediaTek, Apple Silicon), а также в различных бытовых устройствах.

Первая версия архитектуры: ARMv1 – вышла в 1985 году, была 32 разрядной (разрядность регистров), а также имела 26 битное пространство памяти (поддерживалось 2^{26} различных адресов)

Современными считаются следующие версии архитектуры: ARMv7 (32-битная), ARMv8 (64 битная с поддержкой совместимости с ARMv7), а также новая перспективная ARMv9, которая поддерживает векторные (SIMD) инструкции SVE2, улучшающая расширение SVE.

Профили процессоров ARM бывают следующими:

- Application – поддерживает виртуальную память, создан для устройств, требующих высокой производительности (смартфоны, планшеты);
- Microcontroller – отсутствуют блоки управления памятью, создан для микроконтроллеров и недорогих встраиваемых устройств;
- Real-time – поддерживает режим реального времени
- Classic – не реализуется в современных архитектурах, был основным профилем до архитектуры ARMv6

39 Процессоры ARM. Наборы команд. Основные регистры.

ARM – архитектура (семейство) процессоров, альтернатива архитектуре x86-64 и семейству процессоров Intel x86.

Расшифровывается как Advanced RISC Machine (усовершенствованная RISC машина), где RISC – другая архитектура процессоров, расшифровывается как reduced instruction set computer. Отличительной чертой архитектуры, отличающей его от x86-64, является упрощенный набор инструкций.

Используется ARM главным образом в мобильных устройствах (процессоры Qualcomm, MediaTek, Apple Silicon), а также в различных бытовых устройствах.

Существует несколько наборов команд архитектуры ARM:

- Набор команд ARM (A32) – режим, в котором выполняется 32-битный набор команд. Является баабабом, базовым набором команд ARM
- Набор команд Thumb – облегченный набор команд (16 битных), целью которого является уменьшение размера программы (за счет того, что каждая команда занимает меньше памяти). При этом не поддерживается множество операций, возможных в A32
- Thumb-2 – вторая версия набора команд Thumb (16- и 32-битных), включающий улучшенных набор команд (более функциональных), чем Thumb
- A64 – набор команд ARM 64 бит, которая поддерживает 64 битный набор команд, а также имеет режимы совместимости с некоторыми другими наборами команд, например ARM 32 бит. Включает в себя расширенный набор регистров, расширение NEON, реализующее SIMD (single

instruction multiple data) с помощью набора 128 разрядных регистров и инструкций для работы с ними.

Архитектура ARMv8 включает в себя:

- 31 64-битный регистр R0..R30. Доступ осуществляется по алиасам X0..X30 и по алиасам W0..W30 для младших 32-битных половин. При записи значений в регистры W0..W30 старшие половины регистров зануляются, в отличие от x86;
- PC (program counter) - программный счётчик, аналог IP в x86;
- LR (алиас для X30) - ссылочный регистр. Хранит в себе последний адрес возврата;
- SP - указатель стека;
- V0..V31 - 128-битные SIMD-регистры расширения NEON (аналог SSE в x86). Доступ осуществляется по алиасам Q0..Q31 для всех 128 бит и по другим алиасам для младших частей.

40 Процессоры ARM. Основные команды.

ARM – архитектура (семейство) процессоров, альтернатива архитектуре x86-64. Расшифровывается как Advanced RISC Machine (усовершенствованная RISC машина), где RISC – другая архитектура процессоров, расшифровывается как reduced instruction set computer. Отличительной чертой архитектуры, отличающей его от x86-64, является упрощенный набор инструкций.

Используется ARM главным образом в мобильных устройствах (процессоры Qualcomm, MediaTek, Apple Silicon), а также в различных бытовых устройствах.

Архитектура ARMv8 включает в себя базовые команды:

- LDR *регистр, память* – для загрузки из памяти данных в регистр
- STR *регистр, память* – для загрузки данных из регистра в память
- MOV *приемник, источник* – для пересылки данных, где приемник – некоторый регистр, а источник – регистр или непосредственный операнд (например, численная константа)
- ADD/SUB A, B, C – сложение/вычитание ($A = B \pm C$)
- AND/OR A, B, C – побитовое и/или ($A = B \wedge \vee C$)
- b – переход на метку (ветвление, branching). аналог команды безусловного перехода JMP в x86;
- bl - переход на метку с сохранением адреса возврата в LR;
- br - аналог b, но осуществляет переход по адресу из регистра;
- beq, bne, blt, ble, bgt, bge и т.д. - команды условного ветвления. (аналоги jne, jeq, jbl и т.д.)